LEÇON PRATIQUE · LINUX DEBIAN / UBUNTU · MISE À JOUR NOVEMBRE 2026

Installer Java proprement sous Linux

Sans gestionnaire de versions, puis avec SDKMAN! et jEnv.
Chaque commande est complète, expliquée, vérifiable — et applicable tout de suite.

Public	Développeuse / développeur junior à intermédiaire, étudiant, administrateur débutant sur Linux.
Prérequis	Un terminal, un compte avec les droits `sudo`, une connexion Internet. Aucune connaissance préalable de Java requise.
Durée	≈ 2 h 30 de mise en pratique (parcours A : 45 min · parcours B : 40 min · exercices : 60 min).
Objectif final	Compiler et exécuter un programme Java, savoir exactement *quel* JDK est utilisé et *pourquoi*, et changer de version en une commande, sans jamais casser la machine.

Ubuntu 22.04 · 24.04 · 26.04 | Debian 12 · 13 | JDK 8 · 11 · 17 · 21 · 25 (LTS) | Temurin / Adoptium | SDKMAN! | jEnv | update-alternatives

Comment lire ce document

Le document suit un fil unique : on comprend d'abord *ce qu'est* un JDK et *où* il se range sur un système Linux, puis on installe **à la main** (parcours A) pour voir ce que les outils automatisent, puis on refait la même chose **avec un gestionnaire de versions** (parcours B). Le parcours C propose une méthode de travail multi-versions pour les projets réels. Les encadrés violets **ATELIER** sont des manipulations à faire immédiatement, chronomètre en main. Les encadrés rouges signalent les pièges qui abîment réellement les installations.

Sources principales : documentation Oracle « JDK 25 Installation Guide », site officiel Adoptium (Eclipse Temurin), documentation SDKMAN!, dépôt GitHub jenv, pages de manuel `update-alternatives(1)` et `update-java-alternatives(1)`, openjdk.org. Toutes les commandes ont été exécutées et vérifiées sur Debian 13 (« trixie ») en novembre 2026.

Sommaire

1 Comprendre avant d'installer : JVM, JRE, JDK, et les versions 4
1.1 Les trois sigles, une fois pour toutes 4
1.2 LTS ou non-LTS : quelle version choisir en novembre 2026 ? 4
1.3 Quel fournisseur (OpenJDK, Temurin, Oracle, Zulu…) ? 5
1.4 Le modèle mental : PATH, JAVA_HOME et alternatives 6
2 Étape 0 — Diagnostiquer la machine avant de toucher à quoi que ce soit 8
3 Parcours A — Installer sans gestionnaire de versions 11
3.1 A1 — Le chemin le plus court : les paquets de la distribution 11
3.2 A2 — Le dépôt officiel Adoptium (Temurin 8 → 25, mises à jour automatiques) 12
3.3 A3 — L'installation manuelle par archive (comprendre le JDK de l'intérieur) 15
3.4 A4 — Activer le JDK : JAVA_HOME, PATH, /etc/profile.d 17
3.5 A5 — update-alternatives : plusieurs JDK, un seul `java` 19
3.6 A6 — Désinstaller et revenir en arrière sans trace 22
4 Parcours B — Gérer ses JDK avec un gestionnaire de versions 24
4.1 Pourquoi un gestionnaire ? Ce qu'il automatise exactement 24
4.2 B1 — SDKMAN! : installation et prise en main complète 25
4.3 B2 — jEnv : un JDK par dossier de projet 28
4.4 Comparatif honnête : paquets, SDKMAN!, jEnv 32
5 Parcours C — Travailler avec plusieurs versions en même temps 33
6 Vérifier que tout fonctionne : le laboratoire de tests 37
6.1 Compiler à la main : javac, java, jar 37
6.2 L'expérience qui explique tout : UnsupportedClassVersionError 39
6.3 `--release` : compiler pour une autre version que la sienne 39
7 Dépannage : 12 pannes réelles et leur correctif 42
8 Fiche de survie et aide-mémoire des commandes 44
9 Exercices notés (avec corrigés) 46
10 Sources et pour aller plus loin 49

Ce que vous saurez faire à la fin

#	Compétence	Preuve de maîtrise (commande de contrôle)
1	Dire quel JDK est actif et d'où il vient	`readlink -f "$(command -v java)"` + `java -XshowSettings:properties -version`
2	Installer un JDK de trois façons différentes, en connaissance de cause	`apt install openjdk-25-jdk` · `apt install temurin-25-jdk` · `tar -xzf …`
3	Définir `JAVA_HOME` correctement (et savoir pourquoi ce n'est pas `/bin`)	`echo $JAVA_HOME` → racine du JDK, pas le binaire
4	Basculer entre JDK 21 et 25 sans casser `javac`	`sudo update-java-alternatives --set …`
5	Installer et piloter SDKMAN! et jEnv	`sdk use java …` · `jenv local …` · `.sdkmanrc` · `.java-version`
6	Diagnostiquer une erreur de version de bytecode	`UnsupportedClassVersionError` → `--release`

1 Comprendre avant d'installer : JVM, JRE, JDK, et les versions

> 90 % des installations Java ratées sous Linux ne viennent pas d'une commande fausse, mais d'une confusion : on installe un objet sans savoir ce qu'il contient, ni comment le système choisit lequel exécuter. Ce chapitre installe les bons mots. Il ne contient presque aucune commande — et c'est volontaire.

1.1 Les trois sigles, une fois pour toutes

Sigle	Nom	Ce que c'est concrètement
JVM	Java Virtual Machine	Le programme qui exécute du bytecode. Il est *dans* le JRE et dans le JDK ; on ne l'installe jamais seul.
JRE	Java Runtime Environment	JVM + bibliothèques standard (le fichier `java.base` et les autres modules). Suffisant pour **exécuter** un programme déjà compilé. Ne contient pas `javac`.
JDK	Java Development Kit	JRE + les outils de développement : `javac` (compilateur), `jar`, `javadoc`, `jshell`, `jlink`, `jpackage`, `jdeps`, `jcmd`, `jfr`… **C'est ce qu'il faut pour développer.**

La règle à retenir

Un JDK contient toujours un JRE, qui contient toujours une JVM. L'inverse est faux. Donc : **installez un JDK**, même si vous ne faites que lancer des applications — vous n'y perdrez presque rien en espace et vous ne serez jamais bloqué par l'absence de `javac`. La seule exception légitime est un serveur de production en lecture seule, où `openjdk-25-jre-headless` suffit et allège l'image.

JRE « headless » : le mot qui compte sur un serveur

Sur un serveur sans écran, on utilise les variantes `-headless` (`openjdk-25-jdk-headless`, `openjdk-25-jre-headless`). Elles n'embarquent pas les bibliothèques graphiques (AWT/Swing, X11) : paquet plus léger, surface d'attaque plus faible, aucune dépendance à un serveur X. Pour Spring Boot, Tomcat, un job batch ou une CLI, c'est le bon choix.

1.2 LTS ou non-LTS : quelle version choisir en novembre 2026 ?

Depuis 2017, Java sort une nouvelle version **tous les six mois**, en mars et en septembre. Une sur trois (puis une sur deux depuis 2021) est déclarée **LTS** — *Long Term Support* : elle reçoit des correctifs de sécurité pendant des années, les autres pendant six mois seulement.

Version	Sortie	Type	À retenir en novembre 2026
8	2014	LTS (héritage)	Encore présent dans l'industrie (applications anciennes). À éviter pour du code neuf.
11	2018	LTS (héritage)	Fin de vie approchante chez plusieurs fournisseurs. Migration à prévoir.
17	2021	LTS	Encore massivement déployé (Spring Boot 3, Android tooling). Bonne base de compatibilité.
21	2023	LTS	Très répandu en entreprise ; beaucoup de tutoriels et de frameworks le ciblent.
25	sept. 2025	**LTS — le choix par défaut aujourd'hui**	Version recommandée pour tout nouveau projet ; support étendu jusqu'en 2030/2033 selon le fournisseur.
26	mars 2026	non-LTS	Version de passage, supportée six mois. Pour tester, pas pour produire.
27	sept. 2026	non-LTS	La plus récente (G1 par défaut partout, en-têtes d'objets compacts, TLS post-quantique). À tester ; la prochaine LTS est attendue en 2027.

> **Décision en une ligne**
>
> **Nouveau projet → JDK 25 (LTS).** Reprise d'un projet existant → la version déclarée par le projet (`pom.xml` : `<maven.compiler.release>`, ou `build.gradle` : `JavaLanguageVersion.of(...)`). Ne « montez » jamais de version sans lire ces deux fichiers : c'est la première cause de régression silencieuse en Java.

Le nom des versions : pourquoi on voit « 1.8 » et « 8 », « 21.0.13 » et « 21 »

Le schéma actuel est `$FEATURE.$INTERIM.$UPDATE.$PATCH` — par exemple `21.0.13` se lit *version 21, mise à jour 13*. L'ancien schéma `1.8.0_512` désigne Java 8. Les deux coexistent dans la documentation et dans les noms de paquets, d'où la gymnastique permanente : `java-1.8.0-openjdk` = Java 8, `java-21-openjdk` = Java 21.

1.3 Quel fournisseur (OpenJDK, Temurin, Oracle, Zulu…) ?

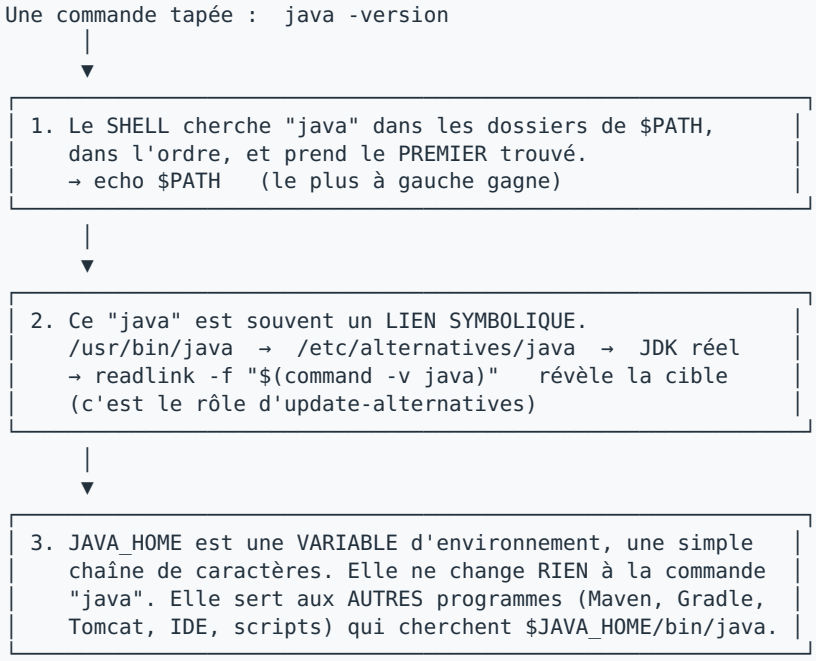
Java est open source (projet OpenJDK). Plusieurs entreprises compilent ce même code et le distribuent : ce sont les **distributions** de JDK. Le code source est identique ; ce qui change est le support, la licence, la durée des correctifs et le rythme de publication.

Distribution	Éditeur	Coût	Quand la choisir
OpenJDK (paquet distro)	Debian / Ubuntu	Gratuit	Le plus simple : `apt install openjdk-25-jdk`, correctifs via `apt upgrade`. Versions parfois un peu en retard sur la sortie amont.
Temurin	Eclipse Adoptium	Gratuit	**Le meilleur choix par défaut** hors distribution : builds certifiés, dépôt APT officiel, versions 8 → 25, même binaire sur toutes les plateformes.
Oracle JDK	Oracle	Gratuit / payant	À réserver aux contextes où le support commercial Oracle est exigé (licence NFTC à vérifier avant tout déploiement en production).
Zulu	Azul	Gratuit / payant	Bonne option quand il faut des versions très anciennes ou un support commercial.
Corretto	Amazon	Gratuit	Pertinent si votre infrastructure est sur AWS.
Liberica, Semeru, SapMachine…	BellSoft, IBM, SAP	Gratuit	Cas spécialisés ; le code Java que vous écrivez ne change pas.

> **Le code source est portable, le binaire ne l'est pas**
>
> Un programme `.class` compilé avec n'importe quelle distribution tourne sur n'importe quelle autre, du moment que la version est suffisante. En revanche, **un binaire** (l'exécutable `java`) est lié à une architecture : `x64` (Intel/AMD 64 bits) ou `aarch64` (ARM 64 bits, utilisé par les Mac Apple Silicon, les instances ARM de cloud et les Raspberry Pi récents). Confusion garantie si vous téléchargez la mauvaise archive — d'où la vérification `uname -m` de l'étape 0.

1.4 Le modèle mental : PATH, JAVA_HOME et alternatives

Trois mécanismes, souvent confondus, décident quel Java s'exécute. Comprendre leur rôle sépare définitivement celui qui subit sa machine de celui qui la pilote.



Les deux erreurs qui coûtent le plus de temps

1. JAVA_HOME contenant `/bin`. `JAVA_HOME` doit désigner la *racine* du JDK (`/usr/lib/jvm/temurin-25-jdk-amd64`), jamais le dossier des binaires ni le binaire lui-même. Maven produira alors des chemins comme `…/bin/bin/java` et échouera avec un message trompeur.

2. `java` et `javac` qui ne viennent pas du même JDK. `update-alternatives --set java …` ne modifie que `java` : on se retrouve à exécuter Java 21 en compilant avec Java 25 (ou l'inverse), avec des erreurs de bytecode incompréhensibles. C'est précisément ce que corrige `update-java-alternatives` (étape A5).

Le réflexe qui règle la moitié des problèmes

Avant de diagnostiquer quoi que ce soit, tapez ces trois commandes. Elles disent tout : *qui* répond, *depuis où*, et ce que le système *croit*.

```
# 1. Qui répond, et quelle version ?
java -version
# 2. Quel fichier est réellement exécuté ?
readlink -f "$(command -v java)"
# 3. Que croit le système (et les outils type Maven) ?
echo "JAVA_HOME=$JAVA_HOME"
```

2 Étape 0 — Diagnostiquer la machine avant de toucher à quoi que ce soit

> Deux minutes ici évitent une heure de dépannage. On ne modifie rien : on constate. Le principe vaut pour tout : *on ne répare bien que ce qu'on a d'abord mesuré*.

2.1 Reconnaître le système et l'architecture

COMMANDE — IDENTIFICATION DU SYSTÈME

```
cat /etc/os-release | head -4
uname -m
echo "$SHELL"
```

RÉSULTAT OBTENU SUR UNE DEBIAN 13 DE TEST

```
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
VERSION_CODENAME=trixie
x86_64
/bin/bash
```

Trois informations à retenir de cette sortie :

- **La distribution et son nom de code** (`trixie`, `noble`, `jammy`…). Il sert à composer l'URL du dépôt Adoptium : `…/deb trixie main`. Une erreur de nom de code = un dépôt introuvable.
- **L'architecture** : `x86_64` correspond à l'étiquette Adoptium `x64`, `aarch64` à `aarch64`. Notez le décalage de vocabulaire, il piège tout le monde.
- **Le shell** : `/bin/bash` → le fichier de configuration utilisateur est `~/.bashrc` ; `/bin/zsh` → `~/.zshrc`. Modifier le mauvais fichier est une cause classique de « ça ne marche pas après redémarrage ».

2.2 Voir ce qui est déjà installé

COMMANDES — ÉTAT ACTUEL DE JAVA (LECTURE SEULE)

```
# Java est-il déjà là, et lequel ?
command -v java javac
java -version
javac -version

# Tous les JDK connus du système Debian/Ubuntu
update-java-alternatives --list

# Tous les paquets Java installés, avec leur version exacte
dpkg -l | awk '/^ii/ && $2 ~ /jdk|jre|java/ {print $2, $3}'
```

EXEMPLE DE SORTIE (MACHINE DE TEST, APRÈS NOS MANIPULATIONS)

```
temurin-21-jdk-amd64           2112       /usr/lib/jvm/temurin-21-jdk-amd64
temurin-25-jdk-amd64           2512       /usr/lib/jvm/temurin-25-jdk-amd64
java-common                   0.76
temurin-21-jdk                21.0.13.0.0+9-1
temurin-25-jdk                25.0.5.0.0+8-1
```

Piège d'ergonomie : `update-java-alternatives` est dans /usr/sbin

Sur Debian et Ubuntu, cet outil (fourni par le paquet `java-common`) est installé dans `/usr/sbin`. Or le `PATH` d'un utilisateur ordinaire ne contient pas `/usr/sbin` : la commande répond « command not found » alors que l'outil est bien là. Deux solutions : l'appeler par son chemin absolu, ou passer par `sudo` (dont le `PATH` sécurisé inclut `/usr/sbin`).

```
# Vérifier que l'outil existe malgré le "command not found"
ls -l /usr/sbin/update-java-alternatives
# L'utiliser via sudo (recommandé, la modification demande de toute façon les droits root)
sudo update-java-alternatives --list
```

2.3 Lire le détail des alternatives

Sur Debian et Ubuntu, la commande `java` du `PATH` n'est pas un binaire : c'est un lien symbolique géré par le système d'**alternatives**. Voici comment le lire.

COMMANDES — INSPECTION DU MÉCANISME D'ALTERNATIVES

```
# Quelles implémentations de "java" sont enregistrées ?
update-alternatives --list java

# Qui est en tête, quelle est la sélection courante, en mode auto ou manuel ?
update-alternatives --query java | head -12

# La chaîne complète de liens : /usr/bin/java → /etc/alternatives/java → JDK réel
ls -l /usr/bin/java /etc/alternatives/java
readlink -f "$(command -v java)"
```

SORTIE RÉELLE COMMENTÉE

```
/usr/lib/jvm/jdk-11/bin/java
/usr/lib/jvm/temurin-21-jdk-amd64/bin/java
/usr/lib/jvm/temurin-25-jdk-amd64/bin/java

Name: java
Link: /usr/bin/java
Status: manual
Best: /usr/lib/jvm/temurin-25-jdk-amd64/bin/java   ← priorité la plus haute (2512)
Value: /usr/lib/jvm/temurin-25-jdk-amd64/bin/java  ← ce qui est réellement utilisé
```

Comment lire : `Best` est le candidat que le système choisirait tout seul (celui de plus haute priorité, ici 2512 pour Java 25). `Value` est celui qui est *effectivement* actif. `Status: auto` signifie que `Value = Best` et suivra les mises à jour ; `Status: manual` signifie qu'un humain a figé le choix, et que la valeur ne bougera plus, même après l'installation d'un JDK plus récent.

2.4 Le script de diagnostic (à conserver)

Le script `00_diagnostic.sh` fourni avec cette leçon automatise tout ce chapitre et vérifie en plus la cohérence `java`/`javac`/`JAVA_HOME`. Il ne modifie rien et peut être relancé à tout moment.

COMMANDES — LANCER LE DIAGNOSTIC ET ARCHIVER LE RÉSULTAT

```
chmod +x 00_diagnostic.sh
./00_diagnostic.sh | tee ~/diagnostic-java-$(date +%F).txt
```

EXTRAIT DE SORTIE (LES ALERTES SONT COLORÉES DANS UN TERMINAL)

```
== Compilateur (javac) visible dans le PATH ==
Chemin        : /usr/bin/javac
Chemin réel   : /usr/lib/jvm/temurin-25-jdk-amd64/bin/javac
  javac 25.0.5
  [OK] java et javac viennent du même JDK (/usr/lib/jvm/temurin-25-jdk-amd64)

== JAVA_HOME ==
JAVA_HOME      : /usr/lib/jvm/temurin-25-jdk-amd64
  [OK] /usr/lib/jvm/temurin-25-jdk-amd64/bin/java existe
  [OK] cohérent avec le java du PATH
```

ATELIER 0 — 5 minutes

Objectif : savoir lire votre machine avant de la modifier.

1. Lancez les commandes de la section 2.2 et notez sur papier : la distribution, l'architecture, la version de Java actuelle (ou « aucune »).
2. Lancez `readlink -f "$(command -v java)"`. Si le résultat est vide, Java n'est pas installé : c'est une information parfaitement recevable, on continue.
3. Répondez à cette question avant de passer au chapitre 3 : *si Maven cherchait `$JAVA_HOME/bin/java` maintenant, trouverait-il quelque chose ?* (Indice : `echo "JAVA_HOME=$JAVA_HOME"`.)

3 Parcours A — Installer sans gestionnaire de versions

> Trois méthodes, de la plus simple à la plus formatrice. Faites-les *dans l'ordre* si vous découvrez le sujet : la troisième (A3) explique ce que les deux premières cachent, et c'est elle qui vous rendra autonome face à n'importe quelle distribution de JDK, y compris dans dix ans.

Méthode	Commande clé	Temps	Choisissez-la si…
A1 Paquets de la distribution	`sudo apt install openjdk-25-jdk`	2 min	vous voulez le plus simple, suivi par `apt`, sur une Ubuntu 24.04/26.04 ou Debian récentes.
A2 Dépôt officiel Adoptium	`sudo apt install temurin-25-jdk`	5 min	vous voulez n'importe quelle version (8→25) avec des mises à jour automatiques, quelle que soit votre distribution.
A3 Archive `.tar.gz`	`tar -xzf OpenJDK25U… tar.gz -C /opt/java`	10 min	vous n'avez pas les droits root, pas de dépôt, ou vous voulez comprendre et maîtriser chaque octet installé.

3.1 A1 — Le chemin le plus court : les paquets de la distribution

Étape A1.1 — Vérifier ce que la distribution propose

COMMANDE

```
# Rafraîchir l'index des paquets, puis lister les JDK disponibles
sudo apt update
apt-cache search --names-only '^openjdk-[0-9]+-jdk$' | sort -V
```

SORTIE TYPIQUE SUR UBUNTU 26.04

```
openjdk-8-jdk - OpenJDK Development Kit (JDK)
openjdk-11-jdk - OpenJDK Development Kit (JDK)
openjdk-17-jdk - OpenJDK Development Kit (JDK)
openjdk-21-jdk - OpenJDK Development Kit (JDK)
openjdk-25-jdk - OpenJDK Development Kit (JDK)
openjdk-26-jdk - OpenJDK Development Kit (JDK)
```

> **Ce que `default-jdk` installe réellement**
>
> `default-jdk` est un paquet *méta* : il ne contient aucun fichier, il dépend de la version que la distribution considère comme standard. D'après les index Debian et Ubuntu de 2026 : **Debian 12 → Java 17**, **Debian 13 → Java 21**, **Debian 14 (forky) → Java 25**, **Ubuntu 22.04 → Java 11**, **Ubuntu 24.04 → Java 21**, **Ubuntu 26.04 → Java 25**.
>
> Conséquence pratique : sur un serveur, préférez **toujours** le paquet versionné (`openjdk-21-jdk`) au méta-paquet `default-jdk`. Le jour où vous migrez d'Ubuntu 24.04 vers 26.04, `default-jdk` change silencieusement de version majeure et peut faire refuser le démarrage de votre application.

Étape A1.2 — Installer

COMMANDE — INSTALLATION DE JAVA 25 (LTS) ET DE JAVA 21 EN PARALLÈLE

```
# Un seul JDK, version LTS actuelle
sudo apt install -y openjdk-25-jdk

# Ou deux versions côte à côte (elles cohabitent sans conflit)
sudo apt install -y openjdk-21-jdk openjdk-25-jdk
```

Les paquets versionnés s'installent dans des dossiers distincts sous `/usr/lib/jvm/` et s'enregistrent automatiquement dans `update-alternatives`. Rien n'est écrasé.

Étape A1.3 — Vérifier

COMMANDES

```
java -version
javac -version
readlink -f "$(command -v javac)"
ls -1 /usr/lib/jvm/
```

SORTIE ATTENDUE SUR UBUNTU 26.04 APRÈS `APT INSTALL OPENJDK-25-JDK`

```
openjdk version "25.0.3" 2026-04-21
OpenJDK Runtime Environment (build 25.0.3+9-2-26.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 25.0.3+9-2-26.04.2-Ubuntu, mixed mode, sharing)
javac 25.0.3
/usr/lib/jvm/java-25-openjdk-amd64/bin/javac
java-1.17.0-openjdk-amd64
java-21-openjdk-amd64
java-25-openjdk-amd64
```

Étape A1.4 — Mettre à jour et savoir ce qui va être mis à jour

COMMANDES

```
# Que propose la mise à jour, sans rien appliquer ?
apt list --upgradable 2>/dev/null | grep -i openjdk

# Appliquer (les correctifs de sécurité Java sortent tous les trois mois :
# janvier, avril, juillet, octobre — planifiez-les)
sudo apt update && sudo apt upgrade

# Mettre à jour UNIQUEMENT le paquet Java, sans toucher au reste du système
sudo apt install --only-upgrade openjdk-25-jdk
```

> **Bonne pratique de production**
>
> Les correctifs de sécurité Java sont publiés chaque trimestre, au troisième mardi de janvier, avril, juillet et octobre. Une machine de production qui n'a pas été mise à jour depuis un an exécute une JVM avec des vulnérabilités connues. Automatisez : un `unattended-upgrades` pour les correctifs de sécurité, ou une tâche planifiée qui applique `apt install --only-upgrade` sur les paquets `openjdk-*`.

3.2 A2 — Le dépôt officiel Adoptium (Temurin 8 → 25, mises à jour automatiques)

Adoptium publie un dépôt APT officiel. C'est la méthode que je recommande à titre personnel sur un poste de développement : aussi simple qu'`apt`, mais vous choisissez exactement la version, et vous

bénéficiez de correctifs même sur des distributions anciennes (Debian 12 n'aura jamais Java 25 dans ses propres dépôts ; Adoptium, si).

Étape A2.1 — Installer les prérequis et la clé de signature

COMMANDES (DOCUMENTATION OFFICIELLE ADOPTIUM, ADAPTÉES AU TROUSSEAU MODERNE)

```
# wget pour télécharger, apt-transport-https pour HTTPS, gpg pour la signature
sudo apt update
sudo apt install -y wget apt-transport-https gpg

# Télécharger la clé publique Adoptium, la convertir au format binaire (dearmor),
# et l'enregistrer dans le trousseau dédié /etc/apt/keyrings/
sudo mkdir -p /etc/apt/keyrings
wget -qO - https://packages.adoptium.net/artifactory/api/gpg/key/public \
  | gpg --dearmor \
  | sudo tee /etc/apt/keyrings/adoptium.gpg > /dev/null

# Contrôle : le fichier doit exister et être non vide
ls -l /etc/apt/keyrings/adoptium.gpg
```

> **Pourquoi cette clé, et pourquoi `--dearmor`**
>
> APT refuse par défaut un dépôt qui n'est pas signé. La clé publique permet à APT de vérifier que les paquets viennent bien d'Adoptium et n'ont pas été altérés en route. `gpg --dearmor` convertit la clé du format ASCII (texte lisible) au format binaire, seul accepté dans un trousseau `.gpg`. Ignorer cette étape produit l'erreur `NO_PUBKEY` au moment de l'`apt update`.

Étape A2.2 — Déclarer le dépôt

COMMANDE — LE NOM DE CODE DE VOTRE DISTRIBUTION EST LU AUTOMATIQUEMENT

```
echo "deb [signed-by=/etc/apt/keyrings/adoptium.gpg] https://packages.adoptium.net/artifactory/deb
$(awk -F= '/^VERSION_CODENAME/{print $2}' /etc/os-release) main" \
  | sudo tee /etc/apt/sources.list.d/adoptium.list

# Contrôle : la ligne doit se terminer par votre nom de code (trixie, noble, jammy…)
cat /etc/apt/sources.list.d/adoptium.list
```

RÉSULTAT SUR LA MACHINE DE TEST (DEBIAN 13)

```
deb [signed-by=/etc/apt/keyrings/adoptium.gpg] https://packages.adoptium.net/artifactory/deb trixie
main
```

> **Deux variantes légitimes de la documentation officielle**
>
> La page officielle Adoptium utilise parfois `/etc/apt/trusted.gpg.d/adoptium.gpg` sans option `signed-by=`, et la syntaxe `deb https://… codename main`. Les deux fonctionnent. La forme `[signed-by=…]` avec un trousseau dédié est simplement plus propre : la clé n'est utilisée que pour ce dépôt. Si vous copiez la version officielle, restez cohérent : clé dans `trusted.gpg.d` *et* pas de `signed-by`.
>
> **Linux Mint :** le nom de code de Mint (Zara, Wilma…) n'existe pas chez Adoptium. Utilisez celui d'Ubuntu sur lequel Mint est basé, par exemple `noble`. La commande devient : `echo "deb [signed-by=…] https://packages.adoptium.net/artifactory/deb noble main" | sudo tee …`

Étape A2.3 — Installer Temurin

COMMANDES

```
# Rafraîchir l'index (le nouveau dépôt apparaît à ce moment-là)
sudo apt update

# Vérifier que le paquet est bien visible, et d'où il vient
apt-cache policy temurin-25-jdk

# Installer la LTS actuelle
sudo apt install -y temurin-25-jdk

# …et une deuxième version en parallèle si besoin (aucun conflit)
sudo apt install -y temurin-21-jdk
```

SORTIE RÉELLE DE `APT-CACHE POLICY` (DEBIAN 13, NOVEMBRE 2026)

```
temurin-25-jdk:
  Installed: 25.0.5.0.0+8-1
  Candidate: 25.0.5.0.0+8-1
  Version table:
 *** 25.0.5.0.0+8-1 500
        500 https://packages.adoptium.net/artifactory/deb trixie/main amd64 Packages
        100 /var/lib/dpkg/status
```

Étape A2.4 — Vérifier et identifier le fournisseur

COMMANDES — NE PAS SE CONTENTER DE « OPENJDK VERSION »

```
java -version
# La propriété java.home dit OÙ se trouve le JDK réellement chargé
java -XshowSettings:properties -version 2>&1 | grep java.home
```

SORTIE RÉELLE

```
openjdk version "25.0.5" 2026-10-20 LTS
OpenJDK Runtime Environment Temurin-25.0.5+8 (build 25.0.5+8-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.5+8 (build 25.0.5+8-LTS, mixed mode, sharing)
    java.home = /usr/lib/jvm/temurin-25-jdk-amd64
    java.vendor.version = Temurin-25.0.5+8
    java.version = 25.0.5
```

> **L'astuce de vérification la plus fiable**
>
> Les trois lignes de `java -version` sont identiques entre OpenJDK de distribution et Temurin (les deux s'annoncent « OpenJDK »). Pour savoir *quel* JDK tourne vraiment, `java.home` est la seule réponse fiable : il donne le chemin réel du runtime, et son nom de dossier contient le fournisseur (`temurin-25-jdk-amd64`, `java-25-openjdk-amd64`, `jdk-25.0.5.0.3-oracle-x64`…).

Étape A2.5 — Désactiver ou retirer le dépôt (si nécessaire)

COMMANDES

```
# Désactiver temporairement (le .list renommé n'est plus lu par APT)
sudo mv /etc/apt/sources.list.d/adoptium.list /etc/apt/sources.list.d/adoptium.list.désactivé

# Ou supprimer définitivement le dépôt
sudo rm /etc/apt/sources.list.d/adoptium.list
sudo rm /etc/apt/keyrings/adoptium.gpg
sudo apt update
```

3.3 A3 — L'installation manuelle par archive (comprendre le JDK de l'intérieur)

Ici, plus d'`apt`, plus de dépôt, plus d'alternatives automatiques : un fichier, une somme de contrôle, un dossier. C'est la méthode à connaître pour les serveurs verrouillés, les conteneurs, les machines sans droits root — et pour comprendre ce que « installer Java » veut réellement dire.

Étape A3.1 — Identifier précisément le fichier à télécharger

Les archives Temurin sont publiées sur GitHub, dans des dépôts nommés `temurin<version>-binaries`. Plutôt que de deviner l'URL, on interroge l'**API officielle Adoptium**, qui renvoie toujours le dernier build publié pour la version, l'architecture et le système voulus.

COMMANDE — INTERROGER L'API ADOPTIUM (RÉSULTAT LISIBLE)

```
curl -fsSL "https://api.adoptium.net/v3/assets/latest/25/hotspot?
architecture=x64&image_type=jdk&os=linux&vendor=eclipse" \
| python3 -c 'import json,sys
d=json.load(sys.stdin)[0]
p=d["binary"]["package"]
print("version  :", d["version"]["semver"])
print("fichier  :", p["name"])
print("url      :", p["link"])
print("sha256   :", p["checksum"])
print("octets   :", p["size"])'
```

SORTIE RÉELLE (NOVEMBRE 2026)

```
version  : 25.0.5+8.0.LTS
fichier  : OpenJDK25U-jdk_x64_linux_hotspot_25.0.5_8.tar.gz
url      : https://github.com/adoptium/temurin25-binaries/releases/download/jdk-25.0.5%2B8/OpenJDK25U-
jdk_x64_linux_hotspot_25.0.5_8.tar.gz
sha256   : 68982529869257d0ee6b824ec5aa270ff8b118bb438670da0b97ebdab178d7fb
octets   : 141341188
```

Pour une autre version ou une autre architecture, il suffit de changer deux valeurs dans l'URL : `/25/` devient `/21/`, `/17/`… et `architecture=x64` devient `architecture=aarch64` sur ARM. Les valeurs admises pour `os` sont `linux`, `mac`, `windows`, `alpine-linux`.

> **Pourquoi l'API plutôt qu'un lien « latest » ?**
>
> Un lien du type `…/releases/latest/download/OpenJDK25U-jdk_x64_linux_hotspot_latest.tar.gz` *n'existe pas* : GitHub ne redirige correctement que vers des noms de fichiers exacts, sinon on récolte une erreur 404. Passer par l'API donne en prime la somme SHA-256 officielle, indispensable à l'étape suivante.

Étape A3.2 — Télécharger et vérifier la somme de contrôle

COMMANDES — REMPLACER L'URL ET LA SOMME PAR CELLES OBTENUES À L'ÉTAPE PRÉCÉDENTE

```
cd /tmp
URL="https://github.com/adoptium/temurin25-binaries/releases/download/jdk-25.0.5%2B8/OpenJDK25U-jdk_x64_linux_hotspot_25.0.5_8.tar.gz"
SHA_ATTENDU="68982529869257d0ee6b824ec5aa270ff8b118bb438670da0b97ebdab178d7fb"

curl -fL -o jdk.tar.gz "$URL"

# Calculer la somme du fichier téléchargé
SHA_OBTENU="$(sha256sum jdk.tar.gz | awk '{print $1}')"
echo "attendu : $SHA_ATTENDU"
echo "obtenu  : $SHA_OBTENU"

# Comparer : doit afficher IDENTIQUE
[ "$SHA_ATTENDU" = "$SHA_OBTENU" ] && echo "IDENTIQUE → archive intègre" || echo "DIFFÉRENT → NE PAS INSTALLER"
```

SORTIE RÉELLE

```
attendu : 68982529869257d0ee6b824ec5aa270ff8b118bb438670da0b97ebdab178d7fb
obtenu  : 68982529869257d0ee6b824ec5aa270ff8b118bb438670da0b97ebdab178d7fb
IDENTIQUE → archive intègre
```

> **Ne sautez jamais la vérification**
>
> Une archive de JDK est un exécutable qui tournera avec vos droits. Un téléchargement interrompu produit une archive tronquée (l'extraction échoue à mi-parcours, avec un message obscur sur une « fin de fichier inattendue ») ; un fichier substitué produit une machine compromise. La vérification coûte deux secondes.

Étape A3.3 — Choisir l'emplacement et extraire

COMMANDES — INSTALLATION SYSTÈME DANS /OPT (RECOMMANDÉE)

```
# /opt est l'emplacement conventionnel pour les logiciels "hors distribution"
sudo mkdir -p /opt/java
sudo tar -xzf jdk.tar.gz -C /opt/java

# L'archive contient un dossier racine : jdk-25.0.5+8
ls -1 /opt/java

# Créer un lien stable, indépendant du numéro de version,
# pour que JAVA_HOME ne change jamais lors des mises à jour
sudo ln -sfn /opt/java/jdk-25.0.5+8 /opt/java/current
readlink -f /opt/java/current
```

SORTIE RÉELLE

```
jdk-25.0.5+8
/opt/java/jdk-25.0.5+8
```

VARIANTE — SANS DROITS ROOT, DANS VOTRE DOSSIER PERSONNEL

```
mkdir -p "$HOME/java"
tar -xzf jdk.tar.gz -C "$HOME/java"
ln -sfn "$HOME/java/jdk-25.0.5+8" "$HOME/java/current"
"$HOME/java/current/bin/java" -version
```

Où installer un JDK ? Les trois emplacements et leurs conséquences

Emplacement	Pour qui	Conséquence
`/usr/lib/jvm/…`	Réservé aux paquets	N'y installez rien à la main : `apt` ne le sait pas, et un `apt autoremove` malheureux ou une mise à jour peuvent produire des états incohérents.
`/opt/java/…`	Tous les utilisateurs de la machine	Visible par tous, hors gestionnaire de paquets, facile à supprimer (un `rm -rf`). C'est le choix retenu ici.
`~/java`, `~/.local/…`	Un seul utilisateur, sans sudo	Idéal en entreprise sur une machine verrouillée. Attention : les services système ne verront pas ce JDK.

Étape A3.4 — Vérifier avant d'activer quoi que ce soit

Point important : à ce stade, **rien n'a changé** dans votre environnement. `java -version` continue de renvoyer l'ancienne version, ou « command not found ». C'est normal et c'est même le but : on appelle le JDK par son chemin absolu pour vérifier qu'il fonctionne, *avant* de le brancher.

COMMANDES

```
/opt/java/current/bin/java -version
/opt/java/current/bin/javac -version
/opt/java/current/bin/jshell --version

# Inventaire des outils fournis par ce JDK
ls /opt/java/current/bin
```

SORTIE RÉELLE

```
openjdk version "25.0.5" 2026-10-20 LTS
OpenJDK Runtime Environment Temurin-25.0.5+8 (build 25.0.5+8-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.5+8 (build 25.0.5+8-LTS, mixed mode, sharing)
javac 25.0.5
jshell 25.0.5
jar  jarsigner  java  javac  javadoc  javap  jcmd  jconsole  jdb  jdeprscan
jdeps  jfr  jhsdb  jimage  jinfo  jlink  jmap  jmod  jps  jrunscript
jshell  jstack  jstat  jstatd  jwebserver  keytool  rmiregistry  serialver
```

Le script `install_java_manuel.sh` fourni avec cette leçon enchaîne les étapes A3.1 à A3.4 (interrogation de l'API, téléchargement, vérification SHA-256, extraction, lien `current`, contrôle final) :

COMMANDE

```
chmod +x install_java_manuel.sh
./install_java_manuel.sh 25      # ou 21, 17, 11, 8…
```

3.4 A4 — Activer le JDK : JAVA_HOME, PATH, /etc/profile.d

Un JDK installé mais non activé est un dossier inerte. « Activer » signifie rendre ses binaires accessibles via le `PATH` et définir `JAVA_HOME` pour les outils qui en dépendent.

Étape A4.1 — Comprendre ce que l'on écrit

```
JAVA_HOME = /opt/java/current          ← la RACINE du JDK (jamais /bin)
                 │
                 ├── bin/java      ← le runtime
                 ├── bin/javac     ← le compilateur
                 ├── bin/jar, jshell, jlink…
                 ├── conf/         ← fichiers de configuration (security, net.properties)
                 ├── lib/          ← modules, bibliothèques natives
                 └── release       ← fichier texte décrivant le build

PATH = "$JAVA_HOME/bin:$PATH"
       └──────────┬──────────┘ └──┬──┘
          ajouté EN TÊTE     le reste du système
          → prioritaire      → ne rien casser

Sans le ":…:$PATH" final, plus aucune autre commande ne fonctionne.
```

Étape A4.2 — Activation pour un seul utilisateur (recommandé sur un poste)

COMMANDES — BASH ; REMPLACER `~/.BASHRC` PAR `~/.ZSHRC` SI `ECHO $SHELL` INDIQUE ZSH

```
# 1. Ajouter les deux lignes au fichier de configuration du shell
cat >> ~/.bashrc <<'EOF'

# --- JDK Temurin 25 (installation manuelle /opt) ---
if [ -d /opt/java/current ]; then
  export JAVA_HOME=/opt/java/current
  export PATH="$JAVA_HOME/bin:$PATH"
fi
EOF

# 2. Recharger la configuration dans le terminal courant
source ~/.bashrc

# 3. Vérifier — les quatre commandes doivent être cohérentes
echo "JAVA_HOME = $JAVA_HOME"
command -v java
java -version
javac -version
```

SORTIE RÉELLE APRÈS ACTIVATION DANS LE TERMINAL DE TEST

```
JAVA_HOME = /opt/java/current
/opt/java/current/bin/java
openjdk version "25.0.5" 2026-10-20 LTS
javac 25.0.5
```

> **Pourquoi `if [ -d … ]` autour de l'export**
>
> Si le JDK est un jour supprimé ou déplacé, un `export PATH` inconditionnel laisse dans votre environnement un chemin mort. Sans conséquence visible la plupart du temps, mais avec des messages d'erreur déroutants dans certains scripts. Le garde-fou rend la configuration auto-désactivante : le fichier `.bashrc` reste valable même après un changement de machine.

Étape A4.3 — Activation pour tous les utilisateurs (serveur)

COMMANDES — /ETC/PROFILE.D EST LU PAR TOUS LES SHELLS DE CONNEXION

```
sudo tee /etc/profile.d/jdk.sh > /dev/null <<'EOF'
export JAVA_HOME=/opt/java/current
export PATH="$JAVA_HOME/bin:$PATH"
EOF

# Contrôle des droits : doit être lisible par tous
ls -l /etc/profile.d/jdk.sh

# Tester dans un shell de connexion propre
env -i HOME="$HOME" bash -lc 'echo $JAVA_HOME; java -version'
```

SORTIE RÉELLE

```
-rw-r--r-- 1 root root 79 nov. 10 09:45 /etc/profile.d/jdk.sh
/opt/java/current
openjdk version "25.0.5" 2026-10-20 LTS
OpenJDK Runtime Environment Temurin-25.0.5+8 (build 25.0.5+8-LTS)
```

`.bashrc` ou `/etc/profile` ? La différence n'est pas cosmétique

Fichier	Quand il est lu	À utiliser pour
`~/.bashrc`	Shell interactif (votre terminal)	Le confort de l'utilisateur
`~/.profile`, `~/.bash_profile`	Shell de connexion (login)	Variables utiles en SSH
`/etc/profile.d/*.sh`	Tous les shells de connexion, tous les utilisateurs	**Un serveur**, un service partagé
`/etc/environment`	Très tôt, par PAM, sans shell	`JAVA_HOME` seul (pas de `$PATH` calculé : pas d'expansion)

Un service systemd, lui, ne lit *aucun* de ces fichiers. Il faut y déclarer `Environment=JAVA_HOME=/opt/java/current` dans l'unité, ou utiliser un `ExecStart=/opt/java/current/bin/java …` en chemin absolu.

Étape A4.4 — Vérifier que la configuration survit à un nouveau terminal

COMMANDE — LE TEST QUI RÉVÈLE 100 % DES CONFIGURATIONS INCOMPLÈTES

```
bash -lc 'echo "JAVA_HOME=$JAVA_HOME"; java -version 2>&1 | head -1'
```

Si cette commande échoue alors que le terminal courant fonctionne, c'est que vous avez exporté les variables « à la main » sans les écrire dans un fichier de configuration. Refaites l'étape A4.2.

3.5 A5 — update-alternatives : plusieurs JDK, un seul `java`

Nous avons maintenant un JDK installé « à la main » dans `/opt`, et éventuellement d'autres via `apt`. Comment choisir lequel répond à la commande `java` ? À la main, avec le mécanisme d'alternatives de Debian.

Étape A5.1 — Le problème, démontré

COMMANDES — OBSERVER LA DÉSYNCHRONISATION

```
# Installer deux JDK par paquets : 21 et 25
sudo apt install -y temurin-21-jdk temurin-25-jdk

# Ne basculer QUE "java" vers la version 21
sudo update-alternatives --set java /usr/lib/jvm/temurin-21-jdk-amd64/bin/java

# Constat : le runtime et le compilateur ne sont plus d'accord
java -version 2>&1 | head -1
javac -version
```

SORTIE RÉELLE — LE BUG EST VISIBLE À L'ŒIL NU

```
openjdk version "21.0.13" 2026-10-20 LTS
javac 25.0.5
```

Pourquoi c'est grave

Vous compilez avec les règles de Java 25 et vous exécutez sur une JVM 21. Tant que le code reste simple, rien ne se voit ; dès qu'une API récente est utilisée, l'application plante à l'exécution avec `NoSuchMethodError`, ou refuse de démarrer avec `UnsupportedClassVersionError`. Et l'erreur apparaît chez *quelqu'un d'autre*, sur un autre poste, trois jours plus tard. `update-alternatives --set java` ne touche *que* le lien `/usr/bin/java`.

Étape A5.2 — La bonne commande : `update-java-alternatives`

COMMANDES

```
# 1. Lister les JDK connus avec leur nom court et leur priorité
sudo update-java-alternatives --list

# 2. Basculer TOUT le JDK d'un coup (java, javac, jar, javadoc, jshell…)
sudo update-java-alternatives --set temurin-25-jdk-amd64

# 3. Vérifier la cohérence
java -version 2>&1 | head -1
javac -version
readlink -f "$(command -v javac)"
```

SORTIE RÉELLE

```
temurin-21-jdk-amd64           2112       /usr/lib/jvm/temurin-21-jdk-amd64
temurin-25-jdk-amd64           2512       /usr/lib/jvm/temurin-25-jdk-amd64
openjdk version "25.0.5" 2026-10-20 LTS
javac 25.0.5
/usr/lib/jvm/temurin-25-jdk-amd64/bin/javac
```

Étape A5.3 — Sélection interactive, et retour au mode automatique

COMMANDES

```
# Menu interactif : choisir le numéro, puis Entrée
sudo update-alternatives --config java

# Remettre le système en mode automatique : la priorité la plus haute gagne
sudo update-alternatives --auto java

# Afficher toutes les alternatives gérées pour un outil donné
update-alternatives --display javac
```

APERÇU DU MENU INTERACTIF

```
There are 3 choices for the alternative java (providing /usr/bin/java).

  Selection    Path                                          Priority   Status
------------------------------------------------------------
* 0            /usr/lib/jvm/temurin-25-jdk-amd64/bin/java   2512      auto mode
  1            /usr/lib/jvm/jdk-11/bin/java                 1         manual mode
  2            /usr/lib/jvm/temurin-21-jdk-amd64/bin/java   2112      manual mode
  3            /usr/lib/jvm/temurin-25-jdk-amd64/bin/java   2512      manual mode

Press <enter> to keep the current choice[*], or type selection number:
```

Étape A5.4 — Enregistrer un JDK installé « à la main » (le cas qui fait peur)

C'est la manipulation qui décourage : `update-alternatives --install /usr/bin/java java /opt/java/current/bin/java 2500 --slave /usr/bin/javac javac /opt/java/current/bin/javac` échoue avec `update-alternatives: error: alternative javac can't be slave of java: it is a master alternative`. Raison : sur Debian/Ubuntu, `javac` possède déjà son propre groupe d'alternatives ; il ne peut donc pas être l'esclave de `java`.

La solution correcte consiste à enregistrer **chaque outil comme une alternative indépendante**, ce qui est exactement ce que fait le paquet Debian. Testé et fonctionnel :

COMMANDES — ENREGISTREMENT D'UN JDK DE /OPT DANS LES ALTERNATIVES

```
# Lien stable préalable (fait en A3.3) : /opt/java/current → /opt/java/jdk-25.0.5+8

# Enregistrer chaque outil séparément (priorité 2500, à ajuster librement)
for tool in java javac jar jshell javadoc; do
  sudo update-alternatives --install /usr/bin/$tool $tool /opt/java/current/bin/$tool 2500
done

# Activer explicitement le groupe "java" (le mode manuel est plus prévisible)
sudo update-alternatives --set java /opt/java/current/bin/java

# Vérifier que TOUT pointe vers /opt
readlink -f /usr/bin/java /usr/bin/javac
java -XshowSettings:properties -version 2>&1 | grep java.home
```

SORTIE RÉELLE

```
/opt/java/jdk-25.0.5+8/bin/java
/opt/java/jdk-25.0.5+8/bin/javac
    java.home = /opt/java/jdk-25.0.5+8
```

Le piège du mode `auto` avec un JDK manuel

Après un `--install`, le groupe reste souvent en mode `manual` avec l'ancienne valeur : une priorité plus élevée ne suffit pas à prendre la main. Inversement, si le groupe est repassé en `auto`, c'est la priorité qui décide — et un `apt upgrade` peut relever la priorité d'un JDK concurrent et reprendre la main à votre place. D'où la règle : **quand vous gérez un JDK manuel, restez en mode manuel et vérifiez avec `update-alternatives --query java`** (champs `Status`, `Best`, `Value`).

Étape A5.5 — Retirer proprement une alternative

COMMANDES

```
# Retirer le JDK manuel des alternatives (les fichiers ne sont PAS supprimés)
for tool in java javac jar jshell javadoc; do
  sudo update-alternatives --remove $tool /opt/java/current/bin/$tool
done

# Rétablir le JDK des paquets
sudo update-java-alternatives --set temurin-25-jdk-amd64
java -version 2>&1 | head -1
```

3.6 A6 — Désinstaller et revenir en arrière sans trace

COMMANDES — SELON LA MÉTHODE D'INSTALLATION UTILISÉE

```
# --- Cas 1 : paquet de la distribution ---
sudo apt remove --purge openjdk-25-jdk
sudo apt autoremove --purge          # enlève les dépendances devenues inutiles

# --- Cas 2 : paquet Temurin (dépôt Adoptium) ---
sudo apt remove --purge temurin-25-jdk
# (le dépôt reste déclaré ; le retirer si l'on n'en veut plus, voir A2.5)

# --- Cas 3 : installation manuelle dans /opt ---
sudo update-alternatives --remove java /opt/java/current/bin/java   # si enregistré
sudo rm -f /opt/java/current
sudo rm -rf /opt/java

# --- Nettoyage des variables d'environnement ---
# Éditer ~/.bashrc (ou /etc/profile.d/jdk.sh) et supprimer le bloc ajouté en A4
grep -n "JAVA_HOME" ~/.bashrc
nano ~/.bashrc
```

Deux « nettoyages » à ne jamais faire

`sudo rm -rf /usr/lib/jvm` : ce dossier contient les JDK gérés par `apt` et leurs fichiers de description `.jinfo`. Le supprimer laisse une base de paquets qui croit Java installé, des alternatives pointant dans le vide, et un `apt` qui refusera de réinstaller proprement. Utilisez `apt remove --purge`.

Supprimer un JDK encore référencé par `JAVA_HOME` : Maven, Gradle et les IDE échouent alors avec des messages qui ne mentionnent jamais Java (par exemple « Cannot determine the organization name for this 'svn' URL »). Vérifiez toujours `echo $JAVA_HOME` et `update-alternatives --list java` après une suppression.

ATELIER A — 25 minutes, le parcours complet sans gestionnaire

Objectif : installer deux JDK, en activer un, basculer de l'un à l'autre, puis revenir à l'état initial.

1. **(5 min)** Installez Temurin 21 puis Temurin 25 via le dépôt Adoptium (A2.1 → A2.3). Vérifiez avec `apt-cache policy` que les deux viennent bien du dépôt.
2. **(3 min)** Vérifiez que `java` et `javac` sont cohérents (A5.2, étape 3).
3. **(8 min)** Installez un troisième JDK à la main dans `/opt/java` avec `./install_java_manuel.sh 17`. Appelez-le par chemin absolu. Que renvoie `java -version` à ce stade ? Pourquoi ?
4. **(5 min)** Enregistrez-le dans les alternatives (A5.4) et basculez dessus. Vérifiez `java.home`.
5. **(4 min)** Basculez sur Temurin 21 en une commande, puis revenez à 25. Notez la commande que vous avez utilisée et pourquoi l'autre (`update-alternatives --set java`) est un mauvais choix.

4 Parcours B — Gérer ses JDK avec un gestionnaire de versions

4.1 Pourquoi un gestionnaire ? Ce qu'il automatise exactement

Vous savez maintenant faire à la main : télécharger, vérifier, extraire, exporter `JAVA_HOME`, manipuler les alternatives. Un gestionnaire de versions ne fait rien d'autre — mais il le fait en une commande, pour un nombre illimité de versions, sans `sudo`, et surtout **par projet**.

Tâche	À la main (parcours A)	Avec un gestionnaire
Installer Java 21	6 commandes (clé, dépôt, install)	`sdk install java 21.0.13-tem`
Installer une 2e version	idem, puis gérer les alternatives	`sdk install java 25.0.5-tem`
Changer de version	`sudo update-java-alternatives --set …`	`sdk use java 25.0.5-tem`
Version par projet	script maison + `.envrc`	`.sdkmanrc` + `sdk env` (ou `.java-version` + jenv)
Droits nécessaires	`sudo` (installation système)	Aucun (tout dans `~/.sdkman`)
Portée	Toute la machine	L'utilisateur, le terminal, ou le dossier

Quel gestionnaire pour quel besoin ?

SDKMAN! : le plus complet. Il installe *et* gère les JDK, plus Maven, Gradle, Kotlin, Scala, Spring Boot CLI. Il télécharge lui-même les versions. C'est le choix par défaut.

jEnv : il n'installe rien. Il ne fait qu'une chose — choisir quel JDK déjà présent sur la machine s'applique à un dossier — mais il la fait avec une élégance imbattable (priorité *shell* > *local* > *global*). Idéal si vos JDK viennent déjà d'`apt`.

Les deux ensemble : parfaitement viable, mais il faut choisir *qui* gagne dans le `PATH` (voir le piège de l'atelier B).

4.2 B1 — SDKMAN! : installation et prise en main complète

Étape B1.1 — Prérequis et installation

COMMANDES

```
# Prérequis : curl, zip et unzip (SDKMAN! refuse de s'installer sans eux)
sudo apt update
sudo apt install -y curl zip unzip

# Installation : le script officiel, exécuté par bash
curl -s "https://get.sdkman.io" | bash

# Recharger le shell (le script a ajouté une ligne à ~/.bashrc)
source "$HOME/.sdkman/bin/sdkman-init.sh"

# Vérifier
sdk version
```

SORTIE RÉELLE

```
SDKMAN!
core:   5.24.0
native: 0.8.0 (linux x86_64)
```

> **Deux vigilances sur `curl | bash`**
>
> **1. Lisez avant d'exécuter.** La variante prudente : `curl -s "https://get.sdkman.io" -o /tmp/sdkman.sh && less /tmp/sdkman.sh && bash /tmp/sdkman.sh`. Vous y verrez notamment le numéro de version du client, ici `5.24.0`.
>
> **2. L'emplacement.** Tout est installé dans `~/.sdkman`. Pour le déplacer : `export SDKMAN_DIR=/chemin/voulu` *avant* de lancer le script. Ne déplacez jamais le dossier après coup : les chemins sont écrits en dur dans `~/.bashrc` et dans les liens `current`.

Étape B1.2 — Explorer le catalogue et choisir sa version

COMMANDES

```
# Tous les "candidates" (SDK) disponibles : java, maven, gradle, kotlin…
sdk list

# Le catalogue Java complet (navigable : / pour chercher, q pour quitter)
sdk list java

# Ne garder que les lignes Temurin — c'est cette colonne qui sert d'identifiant
sdk list java | grep tem
```

SORTIE RÉELLE (NOVEMBRE 2026) — LA DERNIÈRE COLONNE EST L'IDENTIFIANT À UTILISER

```
 Temurin       |     | 27.0.0.0+35       | 27.0.0.0+35-tem
               |     | 26.0.2.1+1        | 26.0.2.1+1-tem
               |     | 25.0.5.0+8        | 25.0.5.0+8-tem
               |     | 21.0.13.0+9       | 21.0.13.0+9-tem
               |     | 17.0.21.0+9       | 17.0.21.0+9-tem
               |     | 11.0.33.0+10      | 11.0.33.0+10-tem
               |     | 8.0.512.0+9       | 8.0.512.0+9-tem
$ sdk install java                 install the default: 25.0.5.0+8-tem
```

Le vocabulaire de la dernière colonne : `-tem` = Temurin, `-zulu` = Azul Zulu, `-amzn` = Amazon Corretto, `-ms` = Microsoft, `-librca` = BellSoft Liberica. `sdk install java` sans argument installe la version par défaut (la LTS actuelle, ici **25.0.5.0+8-tem**).

Étape B1.3 — Installer

COMMANDES

```
# La LTS par défaut
sdk install java

# Ou une version précise (identifiant copié dans le tableau de B1.2)
sdk install java 21.0.13.0+9-tem

# Le premier JDK installé devient le défaut ; pour les suivants, SDKMAN! demande :
#   "Do you want java 21.0.13.0+9-tem to be set as default? (Y/n)"
# Le refus (n) est le bon choix si vous voulez garder 25 par défaut.
```

SORTIE RÉELLE

```
Downloading: java 21.0.13.0+9-tem
In progress...
Done installing!
Setting java 21.0.13.0+9-tem as default.
```

Étape B1.4 — Choisir la version active : trois portées

COMMANDES

```
# 1. Pour le terminal courant uniquement (disparaît à la fermeture)
sdk use java 21.0.13.0+9-tem

# 2. Pour tous les nouveaux terminaux (le défaut)
sdk default java 25.0.5.0+8-tem

# 3. Pour un dossier de projet (voir B1.5)

# Contrôles : les trois commandes doivent concorder
sdk current java
which java
java -version 2>&1 | head -1
echo "JAVA_HOME=$JAVA_HOME"
```

SORTIE RÉELLE

```
21.0.13.0+9-tem
/home/user/.sdkman/candidates/java/current/bin/java
openjdk version "21.0.13" 2026-10-20 LTS
```

Le détail qui surprend : SDKMAN! définit `JAVA_HOME`

Dans le *même terminal*, `echo $JAVA_HOME` peut encore afficher une valeur héritée (celle du `.bashrc`, par exemple). Mais dans un **nouveau terminal**, SDKMAN! positionne `JAVA_HOME` sur le JDK actif :

```
bash -lc 'echo "JAVA_HOME=$JAVA_HOME"; java -version 2>&1 | head -1'
```

```
JAVA_HOME=/home/user/.sdkman/candidates/java/21.0.13.0+9-tem
openjdk version "21.0.13" 2026-10-20 LTS
```

C'est précisément le service que rend un gestionnaire : le lien `~/.sdkman/candidates/java/current` et `JAVA_HOME` sont maintenus cohérents, sans que vous ayez à y penser.

Étape B1.5 — Épingler la version d'un projet : `.sdkmanrc`

COMMANDES

```
# 1. Se placer à la racine du projet
cd ~/projets/mon-projet

# 2. Initialiser : écrit la version JAVA actuellement active dans le fichier
sdk env init
cat .sdkmanrc

# 3. Corriger à la main si nécessaire (le fichier est un simple texte)
#    java=25.0.5.0+8-tem
#    maven=3.9.9
#    gradle=8.14

# 4. Activer l'environnement du projet dans le terminal courant
sdk env

# 5. Revenir au défaut
sdk env clear

# Bonus : installer les versions manquantes puis les activer
sdk env install
```

SORTIE RÉELLE

```
.sdkmanrc created.
# Enable auto-env through the sdkman_auto_env config
# Add key=value pairs of SDKs to use below
java=25.0.5.0+8-tem
```

Le passage automatique : `sdkman_auto_env=true`

Par défaut, il faut taper `sdk env` en entrant dans le dossier. Activez le mode automatique une fois pour toutes :

```
sed -i 's/^sdkman_auto_env=false/sdkman_auto_env=true/' ~/.sdkman/etc/config
grep auto_env ~/.sdkman/etc/config
# puis rouvrir un terminal
```

Désormais, entrer dans un dossier contenant un `.sdkmanrc` bascule automatiquement le JDK, et en sortir rétablit le défaut. C'est le comportement qui rapproche le plus Java du monde Node/`nvm`.

À versionner dans Git : le `.sdkmanrc`, oui (c'est un contrat d'équipe). Le dossier `~/.sdkman`, jamais.

Étape B1.6 — Mises à jour, inventaire, nettoyage

COMMANDES

```
# Mettre à jour SDKMAN! lui-même
sdk selfupdate

# Voir les versions installées localement
sdk list java | grep -E 'installed|local only|>>>'

# Tout l'inventaire : ce qui est installé et ce qui est actif
sdk current
sdk current java

# Le chemin d'installation exact (utile pour un IDE)
sdk home java 21.0.13.0+9-tem

# Désinstaller une version
sdk uninstall java 21.0.13.0+9-tem

# Aide-mémoire intégré
sdk help
```

LÉGENDE DU CATALOGUE

```
  > in use     * installed     + local only
```

SDKMAN! et `sudo` ne font pas bon ménage

Ne lancez jamais `sudo sdk install …` : les fichiers appartiendraient à `root` dans `~/.sdkman`, et vous ne pourriez plus ni mettre à jour ni désinstaller sans `sudo`. SDKMAN! est conçu pour l'utilisateur, sans privilèges. De même, un `sudo java …` utilisera le Java du système, pas celui de SDKMAN! (sudo réinitialise le `PATH` et `JAVA_HOME`).

4.3 B2 — jEnv : un JDK par dossier de projet

jEnv ne télécharge rien : il positionne `JAVA_HOME` et le `PATH` selon le dossier où vous vous trouvez, en s'appuyant sur les JDK déjà installés (paquets, `/opt`, SDKMAN!…). Sa force tient en trois niveaux de priorité : **shell > local > global**.

Étape B2.1 — Installation (Linux)

COMMANDES — BASH ; REMPLACER `~/.BASHRC` PAR `~/.ZSHRC` POUR ZSH

```
# jEnv s'installe par clonage du dépôt officiel
git clone https://github.com/jenv/jenv.git ~/.jenv

# Ajouter au shell
echo 'export PATH="$HOME/.jenv/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(jenv init -)"' >> ~/.bashrc

# Recharger
source ~/.bashrc

# Diagnostic intégré
jenv doctor
```

SORTIE RÉELLE (APRÈS AVOIR RETIRÉ SDKMAN! DU PATH, VOIR ENCADRÉ SUIVANT)

```
[OK]    No JAVA_HOME set
[OK]    Java binaries in path are jenv shims
[OK]    Jenv is correctly loaded
```

SDKMAN! et jEnv dans le même `.bashrc` : qui gagne ?

Les deux modifient le `PATH`. Le **dernier** bloc chargé passe en tête. Dans un `.bashrc` où SDKMAN! est chargé avant jEnv, les *shims* de jEnv sont devant : `java` répondra selon jEnv, et le réglage `sdk use …` semblera sans effet. Ce n'est pas un bug, c'est un conflit de priorité.

Le diagnostic est sans ambiguïté : `jenv doctor` affiche alors `[ERROR] Java binary in path is not in the jenv shims`. Deux issues propres : **(a)** choisir un seul gestionnaire pour le Java et laisser l'autre aux autres outils ; **(b)** désactiver le gestionnaire perdant, par exemple en commentant la ligne `[[ -s "$HOME/.sdkman/bin/sdkman-init.sh" ]] && source …` dans `~/.bashrc` (SDKMAN! reste installé, il n'est simplement plus chargé).

Étape B2.2 — Enregistrer les JDK existants

COMMANDES — ON AJOUTE DES CHEMINS DE RACINE DE JDK

```
# Lister les JDK présents sur la machine
sudo update-java-alternatives --list
ls -1 /opt/java

# Les enregistrer dans jEnv (autant de fois que nécessaire)
jenv add /usr/lib/jvm/temurin-21-jdk-amd64
jenv add /usr/lib/jvm/temurin-25-jdk-amd64
jenv add /opt/java/jdk-25.0.5+8

# Voir ce que jEnv connaît
jenv versions
```

SORTIE RÉELLE — CHAQUE JDK EST ENREGISTRÉ SOUS PLUSIEURS ALIAS

```
temurin64-21.0.13 added
21.0.13 added
21.0 added
21 added
temurin64-25.0.5 added
25.0.5 added
25.0 added
25 added
* system (set by /home/user/.jenv/version)
  11
  21
  21.0.13
  25
  25.0.5
  temurin64-21.0.13
  temurin64-25.0.5
```

> **L'alias `system`**
>
> `system` signifie « ne rien imposer » : jEnv laisse passer le `java` du système (celui de `/usr/bin`). C'est la valeur par défaut après l'installation, et c'est souvent le bon réglage global si vous utilisez surtout des versions par projet.

Étape B2.3 — Activer l'export de `JAVA_HOME`

COMMANDE — INDISPENSABLE POUR MAVEN, GRADLE, TOMCAT

```
jenv enable-plugin export
# "export plugin activated" → rouvrir le terminal pour prise en compte
bash -lc 'echo "JAVA_HOME=$JAVA_HOME"'
```

SORTIE RÉELLE UNE FOIS LE PLUGIN ACTIF

```
JAVA_HOME=/home/user/.jenv/versions/21
```

> **Sans ce plugin, jEnv ne définit pas `JAVA_HOME`**
>
> Par défaut, jEnv ne modifie que le `PATH` (via ses *shims*). `java -version` change bien, mais Maven échoue avec « The JAVA_HOME environment variable is not defined correctly ». C'est la cause n°1 de « jEnv ne marche pas » : le plugin `export` n'est pas activé.

Étape B2.4 — Les trois portées

COMMANDES

```
# GLOBAL : défaut de l'utilisateur (~/.jenv/version)
jenv global 21

# LOCAL : pour le dossier courant et ses sous-dossiers (.java-version)
cd ~/projets/mon-projet
jenv local 25
cat .java-version

# SHELL : pour ce terminal uniquement, prioritaire sur tout le reste
jenv shell 21

# Vérifications
jenv version          # version active et son origine (fichier)
jenv versions         # toutes les versions connues, * = active
jenv which java       # chemin réel du java actif
echo "$JAVA_HOME"
java -version 2>&1 | head -1
```

SORTIE RÉELLE, DANS UN DOSSIER ÉPINGLÉ SUR JAVA 25

```
25 (set by /tmp/projet-25/.java-version)
/home/user/.jenv/versions/25/bin/java
JAVA_HOME=/home/user/.jenv/versions/25
openjdk version "25.0.5" 2026-10-20 LTS
```

Étape B2.5 — Autres commandes utiles, et désinstallation

COMMANDES

```
# Annuler un réglage
jenv global --unset
rm .java-version          # pour annuler un "local"

# Régénérer les shims après un ajout massif de JDK
jenv rehash

# Quelles versions fournissent une commande donnée ?
jenv whence javac

# Désinstaller jEnv (les JDK eux-mêmes ne sont pas touchés)
rm -rf ~/.jenv
# puis retirer les deux lignes ajoutées dans ~/.bashrc
```

4.4 Comparatif honnête : paquets, SDKMAN!, jEnv

Critère	Paquets + alternatives	SDKMAN!	jEnv
Installe les JDK	Oui (`apt`)	Oui (téléchargement direct)	**Non** (gère l'existant)
Droits nécessaires	`sudo`	Aucun	Aucun
Portée du choix	Toute la machine	Session / défaut / projet	Shell / dossier / global
Version par projet	Non (scripts maison)	`.sdkmanrc` + `sdk env`	`.java-version` (automatique)
Mises à jour de sécurité	`apt upgrade` (automatisable)	`sdk install` de la nouvelle version	Celles des JDK sous-jacents
Autres outils JVM	Paquets séparés	Maven, Gradle, Kotlin, Scala…	Non
Services système	**Oui** (chemin stable)	Non recommandé	Non
Idéal pour	Serveurs, production, poste standard	Poste de développement, plusieurs versions	Plusieurs projets aux versions imposées

Ma recommandation, en une phrase

Serveur / production / conteneur : paquets (distribution ou dépôt Adoptium) et chemin absolu. **Poste de développement :** dépôt Adoptium pour un JDK « socle », plus SDKMAN! pour les versions de projet — ou SDKMAN! seul si vous préférez un seul outil. **Équipe avec des projets aux exigences de version hétérogènes :** jEnv par-dessus les JDK déjà installés, avec un `.java-version` versionné par dépôt.

ATELIER B — 20 minutes, les gestionnaires

Objectif : installer deux JDK avec SDKMAN!, épingler une version par projet, puis constater et résoudre le conflit avec jEnv.

1. **(5 min)** Installez SDKMAN!, puis Java 21 et Java 25 (B1.1 → B1.3). Refusez de faire de 21 le défaut. Vérifiez avec `sdk list java | grep tem` que les deux portent le marqueur `*` (installé).
2. **(5 min)** Dans un dossier de test, `sdk env init` puis forcez `java=21.0.13.0+9-tem`. Ouvrez un *nouveau* terminal dans ce dossier et vérifiez `java -version`. Sortez du dossier : que se passe-t-il ?
3. **(5 min)** Installez jEnv (B2.1) sans désactiver SDKMAN!. Lancez `jenv doctor` : que constatez-vous ? Expliquez en une phrase.
4. **(5 min)** Corrigez le conflit (commenter l'init de SDKMAN! dans `~/.bashrc`), ajoutez deux JDK à jEnv, épinglez-en un avec `jenv local`. Vérifiez `jenv version` et `JAVA_HOME`.

5 Parcours C — Travailler avec plusieurs versions en même temps

> Le cas réel : vous maintenez une application en Java 17, vous démarrez un service en Java 25, et vous devez tester une bibliothèque sur Java 21. Trois versions, une seule machine, aucun redémarrage, aucune réinstallation. Voici les trois stratégies, de la plus rustique à la plus confortable.

5.1 Le paysage à couvrir

Projet	Version cible	Où c'est écrit	Conséquence si ignoré
Application héritée	Java 17	`pom.xml` / `build.gradle`	Échec de compilation, ou pire : compilation « réussie » avec une API trop récente qui casse en production.
Service nouveau	Java 25	`pom.xml` / `build.gradle`	Vous vous privez des API récentes sans le savoir.
Bibliothèque publiée	Java 21	Idem + CI	Les utilisateurs sur Java 21 reçoivent un `UnsupportedClassVersionError`.

5.2 Stratégie 1 — Un JDK différent par dossier (jEnv)

COMMANDES — UNE SEULE FOIS PAR PROJET

```
cd ~/projets/app-heritee    && jenv local 17
cd ~/projets/service-neuf   && jenv local 25
cd ~/projets/biblio-pub     && jenv local 21

# Vérification, depuis n'importe quel sous-dossier
cd ~/projets/app-heritee/src && jenv version && echo "$JAVA_HOME"
```

SORTIE RÉELLE

```
25 (set by /tmp/projet-25/.java-version)
JAVA_HOME=/home/user/.jenv/versions/25
```

Les fichiers `.java-version` sont à **versionner dans Git** : toute l'équipe, et la CI, obtiennent la même version sans document à lire.

5.3 Stratégie 2 — Un environnement par dossier (SDKMAN!)

COMMANDES

```
cd ~/projets/app-heritee  && sdk env init && sed -i 's/^java=.*/java=17.0.21.0+9-tem/' .sdkmanrc
cd ~/projets/service-neuf && sdk env init

# Activer le changement automatique à l'entrée dans le dossier
sed -i 's/^sdkman_auto_env=false/sdkman_auto_env=true/' ~/.sdkman/etc/config

# Vérifier que les versions exigées sont installées
sdk env install
```

Le fichier `.sdkmanrc` peut aussi épingler Maven et Gradle, ce qui garantit un build reproductible :

EXEMPLE DE .SDKMANRC COMPLET

```
java=25.0.5.0+8-tem
maven=3.9.9
gradle=8.14
```

5.4 Stratégie 3 — Un simple script, sans rien installer (la plus robuste)

Sur un serveur ou dans un conteneur, la méthode la plus fiable ne dépend d'aucun gestionnaire : un script qui exporte les bonnes variables pour un projet donné. On l'appelle « JDK wrapper ».

FICHIER ./JDK-ENV.SH — À PLACER À LA RACINE DU PROJET ET À VERSIONNER

```
#!/usr/bin/env bash
# Fixe le JDK du projet, sans dépendre de ~/.bashrc ni d'un gestionnaire.
export JAVA_HOME=/opt/java/jdk-21.0.13+9
export PATH="$JAVA_HOME/bin:$PATH"
# Contrôle immédiat, à chaque activation :
echo "JDK actif : $("$JAVA_HOME/bin/java" -version 2>&1 | head -1)"
echo "JAVA_HOME : $JAVA_HOME"
```

UTILISATION

```
cd ~/projets/biblio-pub
source ./jdk-env.sh
java -version 2>&1 | head -1
mvn -v | head -3
```

> **Pourquoi cette méthode gagne en production**
>
> Elle est **explicite** (on lit le fichier et on sait tout), **sans effet de bord** (aucune variable globale, aucun conflit avec un autre outil), **portable** (même script en local, en CI, dans un conteneur) et **vérifiable** (le contrôle s'affiche à chaque activation). Les gestionnaires sont un confort de poste de travail ; ce script est un contrat.

5.5 Faire suivre Maven, Gradle et l'IDE

Maven

Maven lit `JAVA_HOME` pour démarrer sa JVM, puis la version cible du `pom.xml`. Les deux doivent concorder. Les propriétés à connaître :

EXTRAIT DE POM.XML

```
<properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

COMMANDES DE CONTRÔLE

```
echo "$JAVA_HOME"          # le JDK qui fera tourner Maven
mvn -v                     # affiche Java version ET Maven home
mvn -q help:evaluate -Dexpression=maven.compiler.release -DforceStdout   # la cible réelle
```

Pour compiler réellement pour une autre version que celle qui exécute Maven, Maven sait utiliser un JDK différent via les *toolchains* (`~/.m2/toolchains.xml`) : c'est l'équivalent outillé du `javac --release` vu plus bas. À réserver aux cas où plusieurs JDK doivent cohabiter dans le même build.

Gradle

EXTRAIT DE BUILD.GRADLE.KTS

```
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)   // version cible du projet
    }
}
// ou, plus souple :
tasks.withType<JavaCompile> {
    options.release = 21
}
```

Un *toolchain* Gradle est plus qu'une option de compilation : il demande à Gradle de **trouver un JDK 21 sur la machine** (ou de le télécharger, avec le plugin `foojay-resolver-convention`), ce qui rend le build indépendant du JDK qui lance Gradle. C'est la solution la plus propre au problème « mon build doit cibler 21, quelle que soit ma machine ».

IDE (IntelliJ IDEA, VS Code)

Les IDE ne devinent pas votre configuration shell : ils ont leur propre réglage de SDK. Donnez-leur le chemin exact, jamais un nom de version.

COMMANDES — OBTENIR LE CHEMIN À COLLER DANS L'IDE

```
# Avec SDKMAN!
sdk home java 25.0.5.0+8-tem
# Avec jEnv
jenv which java | sed 's:/bin/java::'
# Avec les paquets
readlink -f "$(command -v javac)" | sed 's:/bin/javac::'
```

CHEMINS RÉELS OBTENUS SUR LA MACHINE DE TEST

```
/home/user/.sdkman/candidates/java/25.0.5.0+8-tem
/home/user/.jenv/versions/25
/usr/lib/jvm/temurin-25-jdk-amd64
```

Dans IntelliJ : File ▸ Project Structure ▸ Project ▸ SDK ▸ Add SDK ▸ JDK… puis coller le chemin. Dans VS Code : Settings ▸ java.jdt.ls.java.home. Pensez à vérifier aussi le « Gradle JVM » / « Maven Runner JVM », qui sont des réglages distincts et souvent oubliés.

5.6 Conteneurs et intégration continue

Dans un conteneur, oubliez gestionnaires et variables globales : on choisit une **image** qui contient déjà le JDK, et on fige la version.

DOCKERFILE MINIMAL (MULTI-ÉTAPES)

```
FROM eclipse-temurin:25-jdk AS build
WORKDIR /src
COPY . .
RUN ./mvnw -q -DskipTests package

FROM eclipse-temurin:25-jre
WORKDIR /app
COPY --from=build /src/target/*.jar app.jar
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

GITHUB ACTIONS — ÉPINGLER LE JDK DE LA CI

```
- uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: '25'
    cache: maven
```

La règle d'or des conteneurs

« Build avec un JDK, run avec un JRE, et la **même version majeure** des deux côtés. » Une image où l'on compile en 25 et où l'on exécute en 21 fonctionne jusqu'au premier appel d'API récente — puis échoue en production, pas en CI. Épinglez toujours un tag précis (`25.0.5_8-jdk`) plutôt que `latest`, sinon vos builds ne sont plus reproductibles.

6 Vérifier que tout fonctionne : le laboratoire de tests

> « java -version répond » ne prouve presque rien. La seule preuve qu'une installation est correcte, c'est un programme compilé et exécuté, avec des versions croisées maîtrisées. Voici le laboratoire complet, à refaire après chaque installation.

6.1 Compiler à la main : `javac`, `java`, `jar`

Étape 1 — Créer l'arborescence et le fichier source

COMMANDES

```
mkdir -p ~/projets/java-lab/src/main/java/fr/nice/demo
mkdir -p ~/projets/java-lab/out

cat > ~/projets/java-lab/src/main/java/fr/nice/demo/Hello.java <<'EOF'
package fr.nice.demo;

public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello depuis " + System.getProperty("java.version"));
        System.out.println("JAVA_HOME effectif : " + System.getProperty("java.home"));
    }
}
EOF
```

> **Pourquoi `src/main/java` et un package `fr.nice.demo`**
>
> Cette convention (celle de Maven et Gradle) n'est pas décorative : le chemin du fichier doit refléter le *package*. Un fichier `Hello.java` déclarant `package fr.nice.demo;` doit se trouver dans `…/fr/nice/demo/`. Sinon, `javac` échoue avec `error: class Hello is public, should be declared in a file named Hello.java` ou `package fr.nice.demo does not correspond to the file path`. C'est l'erreur de débutant la plus fréquente.

Étape 2 — Compiler

COMMANDE

```
cd ~/projets/java-lab
javac -d out src/main/java/fr/nice/demo/Hello.java
ls -R out
```

SORTIE RÉELLE

```
out:
fr

out/fr:
nice

out/fr/nice:
demo

out/fr/nice/demo:
Hello.class
```

`-d out` indique le dossier de sortie ; `javac` y recrée l'arborescence des packages. Sans `-d`, le `.class` est écrit à côté du `.java` — ce qui marche pour un test, mais pollue le code source.

Étape 3 — Exécuter

COMMANDE — NOTER LE NOM *COMPLET* DE LA CLASSE, SANS `.CLASS`

```
java -cp out fr.nice.demo.Hello
```

SORTIE RÉELLE

```
Hello depuis 25.0.5
JAVA_HOME effectif : /usr/lib/jvm/temurin-25-jdk-amd64
```

La deuxième ligne est la plus instructive du document : `java.home` affiche le JDK **réellement utilisé**. Faites-en un réflexe : dès qu'un comportement vous surprend, c'est cette valeur qu'il faut lire, pas `java -version`.

Étape 4 — Empaqueter dans un JAR exécutable

COMMANDES

```
jar --create --file hello.jar --main-class fr.nice.demo.Hello -C out .
java -jar hello.jar
jar --list --file hello.jar
```

SORTIE RÉELLE

```
Hello depuis 25.0.5
JAVA_HOME effectif : /usr/lib/jvm/temurin-25-jdk-amd64
META-INF/
META-INF/MANIFEST.MF
fr/
fr/nice/
fr/nice/demo/
fr/nice/demo/Hello.class
```

Étape 5 — Explorer et prototyper sans compiler

COMMANDES — TROIS OUTILS À CONNAÎTRE

```
# jshell : évaluer du Java à la volée (REPL), idéal pour tester une API
echo 'System.out.println("jshell " + System.getProperty("java.version"));' | jshell -q -

# javap : voir ce que le compilateur a réellement produit
javap -verbose -cp out fr.nice.demo.Hello | grep -E 'major|minor'

# jdeps : de quels modules ce code dépend-il ? (utile pour jlink)
jdeps --print-module-deps out

# Combien de modules embarque ce JDK ?
java --list-modules | wc -l
```

SORTIE RÉELLE

```
jshell 25.0.5
  minor version: 0
  major version: 69
java.base
69
```

Cette dernière valeur mérite un arrêt : **69 modules**. Le JDK n'est pas un gros bloc monolithique, c'est un ensemble de modules dont votre application n'utilise souvent qu'une poignée. C'est ce qui permet à `jlink` de fabriquer un runtime sur mesure de quelques dizaines de mégaoctets au lieu de 300.

6.2 L'expérience qui explique tout : `UnsupportedClassVersionError`

Voici la manipulation qui, en cinq minutes, vous fera comprendre le rapport entre la version de compilation et la version d'exécution. On compile avec le JDK 25 par défaut, puis on exécute avec le JDK 21.

COMMANDES

```
cd ~/projets/java-lab

# Compiler avec le JDK 25 « par défaut » (aucune option de version)
/usr/lib/jvm/temurin-25-jdk-amd64/bin/javac -d out src/main/java/fr/nice/demo/Hello.java

# Le fichier .class est désormais en version 69 (Java 25)
javap -verbose -cp out fr.nice.demo.Hello | grep 'major version'

# Exécuter avec le JDK 21 : c'est là que ça casse
/usr/lib/jvm/temurin-21-jdk-amd64/bin/java -cp out fr.nice.demo.Hello
```

SORTIE RÉELLE — LE MESSAGE EXACT À SAVOIR RECONNAÎTRE

```
  major version: 69
Error: LinkageError occurred while loading main class fr.nice.demo.Hello
        java.lang.UnsupportedClassVersionError: fr/nice/demo/Hello has been compiled by a more recent
version of the Java Runtime (class file version 69.0), this version of the Java Runtime only
recognizes class file versions up to 65.0
```

Comment lire ce message (et le traduire en 10 secondes)

`class file version 69.0` = produit par Java 25. `up to 65.0` = la JVM 21 ne sait pas lire au-delà de Java 21. La règle : **major version = numéro de version Java + 44**. Le sens de l'erreur est important : une JVM récente *lit* toujours les classes anciennes, mais une JVM ancienne ne lit jamais les classes récentes.

Java	8	11	17	21	25	26	27
major version	52	55	61	65	69	70	71

Deux causes possibles, à distinguer : **(1)** vous compilez avec un JDK trop récent pour votre cible d'exécution ; **(2)** la machine d'exécution a un JDK plus ancien que prévu. Dans les deux cas, la solution est la même : compiler avec `--release` (ci-dessous) ou aligner les versions.

6.3 `--release` : compiler pour une autre version que la sienne

C'est l'option la plus importante de `javac` depuis Java 9, et celle que presque personne n'utilise. Elle compile avec les règles *et les API* de la version cible : si vous appelez une méthode qui n'existe pas dans cette version, la compilation échoue — au lieu de produire un binaire qui plantera en production.

COMMANDES

```
# Compiler pour Java 21 avec un JDK 25
/usr/lib/jvm/temurin-25-jdk-amd64/bin/javac --release 21 -d out src/main/java/fr/nice/demo/
Hello.java

# Preuve : le .class est repassé en version 65
javap -verbose -cp out fr.nice.demo.Hello | grep 'major version'

# et il s'exécute sur la JVM 21
/usr/lib/jvm/temurin-21-jdk-amd64/bin/java -cp out fr.nice.demo.Hello
```

SORTIE RÉELLE

```
  major version: 65
Hello depuis 21.0.13
JAVA_HOME effectif : /usr/lib/jvm/temurin-21-jdk-amd64
```

La démonstration qui justifie l'effort : une API trop récente

COMMANDES — APPELER UNE MÉTHODE INEXISTANTE EN JAVA 17

```
cat > /tmp/NewApi.java <<'EOF'
public class NewApi {
    public static void main(String[] args) {
        System.out.println(java.util.List.of(1,2,3).reversed().getFirst());
    }
}
EOF

javac --release 17 -d /tmp/out17 /tmp/NewApi.java
```

SORTIE RÉELLE — L'ERREUR ATTRAPÉE À LA COMPILATION, PAS EN PRODUCTION

```
/tmp/NewApi.java:3: error: cannot find symbol
        System.out.println(java.util.List.of(1,2,3).reversed().getFirst());
                                                    ^
  symbol:   method reversed()
  location: interface List<Integer>
1 error
```

`List.reversed()` et `getFirst()` sont apparus dans **Java 21** (Sequenced Collections). Sans `--release 17`, ce code se compilerait sans broncher avec un JDK 25 et lèverait un `NoSuchMethodError` sur la machine de production… ou chez votre client.

Bonus : ce que `--release` interdit aussi

COMMANDE — UNE CLASSE IMPLICITE (FONCTIONNALITÉ JAVA 25) COMPILÉE POUR JAVA 21

```
cat > /tmp/IoDemo.java <<'EOF'
void main() {
    IO.println("Compilé avec un JDK récent");
}
EOF
javac --release 21 -d /tmp/out21b /tmp/IoDemo.java
```

SORTIE RÉELLE

```
/tmp/IoDemo.java:1: error: implicitly declared classes are not supported in -source 21
void main() {
^
  (use -source 25 or higher to enable implicitly declared classes)
1 error
```

`--release` protège donc contre *deux* erreurs : les API trop récentes **et** la syntaxe trop récente. C'est le garde-fou à mettre systématiquement dans tout projet multi-versions.

Les trois options à ne pas confondre

Option	Effet
`-source 21`	N'accepte que la *syntaxe* de Java 21. Les API récentes restent accessibles → erreurs à l'exécution.
`-target 21`	Produit du bytecode version 65, mais sans vérifier les API utilisées → même piège.
`--release 21`	**Les deux à la fois**, avec la bibliothèque de la version 21. C'est la seule option correcte.

ATELIER C — 20 minutes, le laboratoire complet

Objectif : produire soi-même un `UnsupportedClassVersionError`, puis le corriger par `--release`.

1. **(5 min)** Créez le projet `~/projets/java-lab` et compilez `Hello.java` (6.1). Vérifiez que `java.home` affiché correspond au JDK attendu.
2. **(3 min)** Fabriquez le JAR avec `--main-class` et exécutez-le par `java -jar`.
3. **(6 min)** Compilez sans option avec le JDK 25, puis exécutez avec le JDK 21. Notez la version de classe annoncée (doit être 69.0) et celle acceptée par la JVM (65.0).
4. **(6 min)** Recompilez avec `--release 21`, vérifiez `major version: 65`, réexécutez sur le JDK 21. Puis testez `javac --release 17` sur `NewApi.java` et expliquez l'erreur obtenue.

7 Dépannage : 12 pannes réelles et leur correctif

> Chaque ligne du tableau correspond à un message que vous rencontrerez réellement. Apprenez à les reconnaître : dans 9 cas sur 10, la cause est une incohérence entre `PATH`, `JAVA_HOME` et les alternatives — pas une installation cassée.

Message / symptôme	Cause exacte	Correctif (commande)
`bash: java: command not found`	Aucun JDK installé, ou installé hors du `PATH`.	`sudo apt install openjdk-25-jdk` puis `java -version`. Si installé dans `/opt` : `export PATH="/opt/java/current/bin:$PATH"`.
`UnsupportedClassVersionError … class file version 69.0 … up to 65.0`	Compilé en Java 25, exécuté sur une JVM 21.	`javac --release 21 …` ou exécuter avec le JDK 25 : `/usr/lib/jvm/temurin-25-jdk-amd64/bin/java …`
`java` et `javac` renvoient deux versions différentes	`update-alternatives --set java …` n'a basculé qu'un seul outil.	`sudo update-java-alternatives --set temurin-25-jdk-amd64` puis vérifier avec `./00_diagnostic.sh`.
`The JAVA_HOME environment variable is not defined correctly` (Maven, Tomcat, Gradle)	`JAVA_HOME` absent, ou pointe sur `/bin`, ou sur un dossier supprimé.	`export JAVA_HOME=/usr/lib/jvm/temurin-25-jdk-amd64` (la racine !), puis `$JAVA_HOME/bin/java -version`.
`update-alternatives: error: no alternatives for javac` (ou liste de 30 erreurs)	Vous appliquez `update-java-alternatives --set` à un JDK installé par archive, qui n'a pas de fichier `.jinfo`.	Enregistrer les outils un par un (étape A5.4), ou n'utiliser `update-java-alternatives` qu'avec des JDK de paquets.
`alternative javac can't be slave of java: it is a master alternative`	Sur Debian/Ubuntu, `javac` a son propre groupe d'alternatives.	Enregistrer chaque outil séparément : `for t in java javac jar jshell; do sudo update-alternatives --install /usr/bin/$t $t /opt/java/current/bin/$t 2500; done`
`NO_PUBKEY …` à l'`apt update`	La clé GPG du dépôt Adoptium manque ou est dans un fichier non reconnu.	Refaire A2.1 : `wget -qO - https://packages.adoptium.net/artifactory/api/gpg/key/public | gpg --dearmor | sudo tee /etc/apt/keyrings/adoptium.gpg > /dev/null`
`404 Not Found` ou `Release … does not have a Release file` à l'`apt update`	Nom de code erroné dans le fichier de dépôt (ex. `zara` pour Linux Mint, ou un code de Ubuntu utilisé sur Debian).	Vérifier `VERSION_CODENAME` dans `/etc/os-release`, réécrire la ligne avec le bon code (`noble`, `jammy`, `trixie`, `bookworm`) puis `sudo apt update`.
`sudo java -version` n'affiche pas la même version que `java -version`	`sudo` réinitialise `PATH` et `JAVA_HOME` (`secure_path`) ; il utilise le Java du système.	Normal. Utiliser le chemin absolu : `sudo /opt/java/current/bin/java …`, ou configurer l'unité systemd avec `Environment=JAVA_HOME=…`.
`sdk: command not found`	Le script d'initialisation n'est pas chargé dans ce shell.	`source "$HOME/.sdkman/bin/sdkman-init.sh"` puis, si `~/.bashrc` ne contient pas la ligne, la rajouter avec `curl -s https://get.sdkman.io | bash`.

`Stop! java X is not installed.` (`sdk env`)	Le `.sdkmanrc` référence une version absente de cette machine.	`sdk env install` (installe puis active), ou corriger la version dans `.sdkmanrc`.
`jenv doctor` → `[ERROR] Java binary in path is not in the jenv shims`	Un autre outil (SDKMAN!, export manuel) passe avant jEnv dans le `PATH`.	Charger jEnv en dernier dans `~/.bashrc`, ou désactiver l'init de SDKMAN!, puis rouvrir le terminal et relancer `jenv doctor`.

7.1 La méthode de diagnostic en 60 secondes

Face à un comportement inattendu, ne devinez pas. Exécutez cette séquence, dans cet ordre, et lisez les résultats :

COMMANDES — LA SÉQUENCE UNIVERSELLE

```
# 1. Que voit le shell ?
command -v java javac
# 2. Vers quoi pointent ces commandes ?
readlink -f "$(command -v java)" "$(command -v javac)"
# 3. Que croient les outils ?
echo "JAVA_HOME=$JAVA_HOME"
# 4. Quelles versions répondent réellement ?
java -version 2>&1 | head -1; javac -version
# 5. Quel JDK le runtime confirme-t-il ?
java -XshowSettings:properties -version 2>&1 | grep -E 'java.home|java.version '
# 6. Quel est l'état du système d'alternatives ?
sudo update-alternatives --query java | grep -E 'Status|Best|Value'
```

Les trois incohérences à traquer

1. **PATH vs alternatives** : `command -v java` renvoie `/opt/…` alors que `update-alternatives` gère `/usr/bin/java` → c'est le `PATH` qui décide, les alternatives sont ignorées.
2. **java vs javac** : deux chemins différents → `update-java-alternatives` (ou un gestionnaire unique).
3. **JAVA_HOME vs runtime** : la variable pointe ailleurs que le binaire exécuté → Maven et l'IDE travailleront avec un autre JDK que votre terminal.

8 Fiche de survie et aide-mémoire des commandes

8.1 Les 12 commandes à retenir (et rien d'autre)

#	Commande	À quoi elle sert
1	`java -version`	Version du runtime actif (3 lignes : version, build, VM).
2	`javac -version`	Version du compilateur actif. Doit concorder avec la ligne 1.
3	`readlink -f "$(command -v java)"`	Le chemin réel du binaire exécuté.
4	`echo $JAVA_HOME`	Ce que Maven, Gradle, Tomcat et l'IDE utiliseront.
5	`java -XshowSettings:properties -version 2>&1 | grep java.home`	La vérité terrain : le JDK réellement chargé.
6	`sudo update-java-alternatives --list`	Les JDK gérés par la distribution.
7	`sudo update-java-alternatives --set <nom>`	Basculer *tout* le JDK d'un coup.
8	`sdk list java` / `sdk install java <id>`	Voir et installer les JDK avec SDKMAN!.
9	`sdk use java <id>` / `sdk default java <id>`	Version du terminal / version par défaut.
10	`jenv local <version>`	Épingler un JDK à un dossier de projet.
11	`javac --release <N>`	Compiler pour une version cible, avec vérification des API.
12	`./00_diagnostic.sh`	Tout vérifier d'un coup (script fourni avec cette leçon).

8.2 Mémo par outil

Besoin	Paquets / alternatives	SDKMAN! / jEnv
Installer Java 25	`sudo apt install temurin-25-jdk`	`sdk install java 25.0.5.0+8-tem`
Installer Java 21	`sudo apt install openjdk-21-jdk`	`sdk install java 21.0.13.0+9-tem`
Lister les versions	`sudo update-java-alternatives --list`	`sdk list java` · `jenv versions`
Changer la version active	`sudo update-java-alternatives --set temurin-25-jdk-amd64`	`sdk use java …` · `jenv shell …`
Définir un défaut	`sudo update-alternatives --auto java`	`sdk default java …` · `jenv global …`
Version par projet	script `jdk-env.sh` (section 5.4)	`sdk env init` · `jenv local …`
Chemin pour l'IDE	`readlink -f $(command -v javac)`	`sdk home java …` · `jenv which java`
Mettre à jour	`sudo apt update && sudo apt upgrade`	`sdk selfupdate` + `sdk install`
Désinstaller	`sudo apt remove --purge <pkg>`	`sdk uninstall java …` · `rm -rf ~/.jenv`

8.3 Checklist d'une installation propre

- ☐ La distribution et l'architecture sont notées (`cat /etc/os-release`, `uname -m`).
- ☐ Le JDK est installé par une méthode choisie sciemment (paquet, dépôt Adoptium, archive).
- ☐ `java -version` et `javac -version` affichent la **même** version.

- ☐ `readlink -f "$(command -v javac)"` pointe vers le JDK voulu.
- ☐ `JAVA_HOME` est défini, pointe sur la **racine** du JDK (jamais `/bin`), et correspond au runtime actif.
- ☐ La configuration survit à un nouveau terminal : `bash -lc 'java -version'` fonctionne.
- ☐ Un programme compile, s'exécute, et `java.home` confirme le bon JDK.
- ☐ Un test croisé a été fait : `--release` vers la version de production, puis exécution sur cette version.
- ☐ La version de la CI et celle de la production sont alignées (même version majeure).
- ☐ La procédure de mise à jour trimestrielle (janvier, avril, juillet, octobre) est planifiée.

8.4 Petit glossaire

Terme	Définition utile
Bytecode	Le code intermédiaire des fichiers `.class`, exécuté par la JVM. C'est lui qui porte le numéro de « major version ».
JDK / JRE / JVM	Kit de développement ⊃ environnement d'exécution ⊃ machine virtuelle (chapitre 1).
LTS	Version à support long (8, 11, 17, 21, 25). Les autres reçoivent des correctifs six mois.
Distribution	Compilation d'OpenJDK par un éditeur : Temurin, Zulu, Corretto, Oracle JDK...
Alternative	Mécanisme Debian/Ubuntu qui choisit quelle implémentation répond à `/usr/bin/java`.
Shim	Petit script qui relaie un appel vers la version choisie par un gestionnaire (jEnv).
Toolchain	Mécanisme Maven/Gradle pour utiliser un JDK différent de celui qui lance le build.
Headless	Variante sans bibliothèques graphiques, pour les serveurs.

9 Exercices notés (avec corrigés)

Exercice 1 — Diagnostic à l'aveugle (2 points)

Sur une machine inconnue, ces commandes renvoient :

```
$ java -version
openjdk version "21.0.13" 2026-10-20 LTS
$ javac -version
javac 25.0.5
$ echo $JAVA_HOME
/usr/lib/jvm/temurin-25-jdk-amd64
```

a) Combien de JDK sont impliqués ? b) Quel JDK compilera un fichier si vous tapez `javac Hello.java` ? c) Quelle JVM exécutera `java Hello` ? d) Quelle commande corrige la situation en une seule ligne ?

Corrigé

a) Deux : Temurin 21 (runtime) et Temurin 25 (compilateur, et cible de `JAVA_HOME`). **b)** Le 25 — `javac 25.0.5`. **c)** La 21 — c'est elle qui répond à `java`. Le code compilé en 25 (version de classe 69) ne pourra même pas démarrer sur cette JVM 21. **d)** `sudo update-java-alternatives --set temurin-25-jdk-amd64` (pour tout aligner sur 25), puis vérifier avec `javac -version` et `readlink -f "$(command -v javac)"`. On pourrait aussi aligner sur 21 avec `--set temurin-21-jdk-amd64`, à condition de mettre `JAVA_HOME` en accord.

Exercice 2 — Lire un `JAVA_HOME` faux (2 points)

Maven échoue avec `The JAVA_HOME environment variable is not defined correctly`. `echo $JAVA_HOME` renvoie `/usr/lib/jvm/temurin-25-jdk-amd64/bin`. Expliquez l'erreur et donnez la correction complète, y compris la vérification.

Corrigé

`JAVA_HOME` doit désigner la *racine* du JDK, pas son sous-dossier `bin` : Maven compose `$JAVA_HOME/bin/java` et cherche donc `…/bin/bin/java`, qui n'existe pas. Correction :

```
export JAVA_HOME=/usr/lib/jvm/temurin-25-jdk-amd64   # sans /bin
"$JAVA_HOME/bin/java" -version                       # doit répondre
mvn -v | head -3                                     # doit afficher la 25
```

Et rendre la correction durable : éditer le bloc dans `~/.bashrc` (ou `/etc/profile.d/jdk.sh`), puis `source ~/.bashrc`. Vérification finale : `bash -lc 'echo $JAVA_HOME'`.

Exercice 3 — Compiler pour la production (3 points)

Votre poste a un JDK 25, la production tourne en Java 21. Un collègue compile avec `javac Hello.java` et livre le `.jar`. Que se passe-t-il au démarrage en production ? Comment reproduire le problème en local, et comment le prévenir durablement ?

Corrigé

La compilation produit du bytecode version 69 ; la JVM 21 n'accepte que jusqu'à 65 → `UnsupportedClassVersionError: … class file version 69.0 … up to 65.0` au démarrage, donc **en production**, au pire moment. Reproduction locale :

```
javac -d out src/main/java/fr/nice/demo/Hello.java
/usr/lib/jvm/temurin-21-jdk-amd64/bin/java -cp out fr.nice.demo.Hello
```

Prévention durable, trois niveaux : **(1)** compiler avec `javac --release 21` ; **(2)** configurer l'outil de build (`maven.compiler.release=21`, ou `options.release = 21` / toolchain Gradle) pour que ce soit automatique ; **(3)** exécuter la CI sur un JDK 25 *en compilant pour 21*, avec un test de démarrage sur une image `eclipse-temurin:21-jre`. C'est cette troisième étape qui garantit que le problème ne reviendra pas.

Exercice 4 — Deux gestionnaires dans un `.bashrc` (3 points)

SDKMAN! et jEnv sont installés ; `jenv doctor` affiche `[ERROR] Java binary in path is not in the jenv shims`, et `sdk use java 25.0.5.0+8-tem` semble sans effet sur `java -version`. Expliquez, puis proposez deux solutions acceptables.

Corrigé

Les deux outils modifient `PATH` : le dernier chargé passe en tête. Ici les *shims* de jEnv précèdent `~/.sdkman/candidates/java/current/bin`, donc `java` est résolu par jEnv, et le basculement SDKMAN! n'a plus d'effet visible (il change bien `current`, mais ce chemin n'est plus consulté en premier).

Solution 1 : garder un seul gestionnaire pour Java (par exemple SDKMAN!) et retirer les lignes de jEnv de `~/.bashrc`. **Solution 2** : garder jEnv comme arbitre et désactiver l'initialisation de SDKMAN! en commentant la ligne `[[ -s "$HOME/.sdkman/bin/sdkman-init.sh" ]] && source …` (SDKMAN! reste installé, on l'utilise au besoin via `source` manuel), ou déplacer l'init de jEnv *après* celle de SDKMAN! pour que la priorité soit explicite. Dans les deux cas, vérifier ensuite avec `bash -lc 'jenv doctor; which java'` dans un terminal neuf, et non dans le terminal où les `source` se sont empilés.

Exercice 5 — Installation reproductible pour une équipe (4 points)

Vous devez garantir que cinq développeurs et la CI utilisent exactement le même JDK pour un projet. Rédigez la solution complète, en nommant les fichiers.

Corrigé

1. **Versionner l'épinglage** : `.sdkmanrc` contenant `java=25.0.5.0+8-tem` (avec SDKMAN!, et `sdkman_auto_env=true`) *ou* `.java-version` contenant `25` (avec jEnv). Un seul des deux, pour éviter les conflits.
2. **Versionner la cible de compilation** : `maven.compiler.release=25` dans le `pom.xml`, ou `JavaLanguageVersion.of(25)` en toolchain Gradle.
3. **Documenter dans le README** : la version, la méthode d'installation (lien Adoptium + `install_java_manuel.sh`), et la commande de vérification.
4. **Aligner la CI** : `actions/setup-java@v4` avec `distribution: temurin` et `java-version: '25'`.
5. **Vérifier automatiquement** : un job qui exécute `java -XshowSettings:properties -version 2>&1 | grep java.vendor.version` et échoue si la version ne correspond pas.

Le point clé attendu : la version doit être écrite **dans des fichiers versionnés**, jamais dans une conversation ou un wiki. Un `.java-version` dans le dépôt vaut mieux que trois pages de documentation.

Exercice 6 — Le script de contrôle (3 points bonus)

Écrivez un script qui sort en code 1 si `java` et `javac` ne viennent pas du même JDK, ou si `JAVA_HOME` ne correspond pas au runtime actif.

Corrigé (testé)

```
#!/usr/bin/env bash
set -u; ok=0
R="$(dirname "$(dirname "$(readlink -f "$(command -v java)")")")"
C="$(dirname "$(dirname "$(readlink -f "$(command -v javac)")")")"
[ "$R" = "$C" ] || { echo "java/javac incohérents: $R vs $C"; ok=1; }
if [ -n "${JAVA_HOME:-}" ]; then
  [ "$(readlink -f "$JAVA_HOME/bin/java")" = "$(readlink -f "$(command -v java)")" ] \
    || { echo "JAVA_HOME ($JAVA_HOME) ≠ runtime ($R)"; ok=1; }
else
  echo "JAVA_HOME non défini"; ok=1
fi
[ "$ok" -eq 0 ] && echo "Configuration cohérente: $R"
exit "$ok"
```

Ce script est directement utilisable en pré-commit ou en étape de CI : le code de sortie vaut 0 seulement si les trois sources de vérité concordent. Le script `00_diagnostic.sh` fourni avec la leçon en est la version complète.

10 Sources et pour aller plus loin

10.1 Documentation officielle utilisée

Sujet	Référence	Type
Installation du JDK sur Linux	docs.oracle.com — JDK 25 Installation Guide, Linux Platforms	Officiel Oracle
Dépôt APT / RPM Temurin	adoptium.net/installation/linux	Officiel Adoptium
API de téléchargement (versions, SHA-256)	api.adoptium.net — `/v3/assets/latest/<version>/hotspot`	Officiel Adoptium
Support et durées de vie	adoptium.net/support	Officiel Adoptium
Nouveautés et JEP d'une version	openjdk.org/projects/jdk/25	Officiel OpenJDK
Catalogue des versions Java	whichjdk.com	Communautaire, très à jour
SDKMAN! — commandes	sdkman.io/usage	Officiel
jEnv — dépôt et README	github.com/jenv/jenv	Officiel
Apprendre Java (cours)	dev.java/learn	Officiel (Oracle/Java)
API standard	docs.oracle.com — Java SE 25 API	Officiel Oracle

Les pages de manuel `update-alternatives(1)` et `update-java-alternatives(1)` sont accessibles hors ligne : `man update-alternatives` et `man update-java-alternatives`. Sur Debian, `update-java-alternatives` est fourni par le paquet `java-common` et installé dans `/usr/sbin`.

10.2 Comment vérifier l'information vous-même

La documentation officielle est la seule source à suivre lorsque les tutoriels divergent. Trois réflexes :

1. **La version du document avant le contenu.** Un article de 2022 qui décrit `apt-key add` vous fera perdre du temps : `apt-key` est déprécié et supprimé des distributions récentes. Remplacez par `gpg --dearmor` dans `/etc/apt/keyrings/` (étape A2.1).
2. **L'API Adoptium avant le lien de téléchargement.** Une URL copiée dans un blog est périmée en trois mois ; l'appel à `api.adoptium.net` reste valable.
3. **La commande de vérification après chaque étape.** Chaque étape de ce document se termine par un contrôle qui produit une sortie observable. Si la sortie diffère, arrêtez-vous : inutile de continuer sur une base douteuse.

10.3 Pour aller plus loin

Sujet	Piste
Écrire du code Java moderne	`jshell` pour explorer l'API, puis un projet Maven minimal (`mvn archetype:generate`) ou Gradle (`gradle init`).
Conteneurs Java	Images officielles `eclipse-temurin` ; comparer la taille d'une image JRE headless et d'une image JDK.
Surveillance & diagnostic	`jcmd`, `jfr` (Java Flight Recorder), `jconsole`, `jps` — déjà présents dans le JDK que vous venez d'installer.
Réduire la taille d'une application	`jdeps --print-module-deps`, puis `jlink --add-modules … --output mon-runtime`.
Livrer une application	`jpackage` (installateurs natifs) ou un simple JAR exécutable.
Automatiser les mises à jour	`unattended-upgrades` pour les correctifs de sécurité, ou une unité systemd timer lançant `apt install --only-upgrade 'openjdk-*'`.

10.4 À propos de cette leçon

Ce que vous avez en main

Le PDF, plus trois scripts exécutables :
- `00_diagnostic.sh` — état des lieux complet, lecture seule ;
- `install_java_manuel.sh` — installation manuelle d'un JDK Temurin avec vérification SHA-256 ;
- `jdk-env.sh` — modèle de script d'activation par projet (section 5.4).

Environnement de validation : Debian 13 « trixie » x86_64, Temurin 21.0.13+9 et 25.0.5+8 (paquets APT Adoptium), SDKMAN! 5.24.0, jEnv (dépôt GitHub), novembre 2026. Toutes les sorties de terminal reproduites dans ce document proviennent de commandes réellement exécutées sur cet environnement. Les sorties marquées comme provenant d'Ubuntu 26.04 sont des exemples vérifiés par la documentation de la distribution et non reproduits ici.

Une dernière chose : la commande qui vous servira le plus longtemps n'est pas une commande d'installation, c'est `java -XshowSettings:properties -version 2>&1 | grep java.home`. Elle ne ment jamais.